

数学学术展评活动答辩报告

平面点集连边中的 三角形计数极值问题

——四问递进的构造、下界与计算验证

向明中学

诺克萨斯队

成员：金煜锋 周舒菡 褚家铭

2026 年 8 月

摘要

通过对平面点集连边问题的系统分析, 本文确立了以公共邻居判据为核心的理论框架: 新增一条边的三角形增量恰等于其端点公共邻居数, 由此导出**完全图最优**与**大团优先**两条构造原则。基于该框架, 本文解决了从有限点集到一般参数、从自由连边到最小度约束的完整极值问题:

- 对 $n = 9, 10$ 给出了 $E(n, k)$ 的完整取值表与最优构造 (第一问);
- 对一般 $n \geq 11$ 给出了 $E(n, k)$ 的精确公式与量级估计 $E(n, k) = \Theta(k^{2/3})$ (第二问);
- 对附加最小度约束 $\delta \geq d$ 的情形给出六项纯理论下界、极端情形闭式公式与计算机辅助求解方案 (第三问);
- 对连通且三角形边不交的极值函数 $F(n, k)$ 给出普适下界 $\max\{3k, n - 1 + k\}$ 、显式构造、精确公式与 CP-SAT 数值验证 (第四问)。

本版本完整给出四问的解答: 前两问与第三问的六项下界、极端闭式与 $n = 9$ 精确表均为纯理论结果; 第四问在 $n \geq \max\{9, 2k + 1\}$ 范围内精确解决, 剩余区域给出普适下界、无解判据的必要条件与 CP-SAT 数值证据 (严格证明列为开放问题)。本文所有理论下界均经计算程序的交叉验证: $n = 9$ 的完备枚举、 $n = 10$ 与 $n \geq 11$ 的 SAT (布尔可满足性) 验证, 并用独立的 OR-Tools CP-SAT 求解器做了第二路复核, 实现理论结果与计算结果的双重印证。

目录

引言	3
问题陈述	3
几何条件与图论框架的等价性	3
记号与约定	3
本文处理的四个问题	4
1 第一问: $n = 9, 10$ 时的完整取值表	5
1.1 问题建模	5
1.2 $n = 9$ 的结果	5
1.2.1 k 的取值范围	5
1.2.2 完备枚举方法	5
1.2.3 $E(9, k)$ 精确值	5
1.3 $n = 10$ 的结果	5
1.3.1 低区间: $1 \leq k \leq 84$	5
1.3.2 高区间: $85 \leq k \leq 120$	7
1.4 第一问答案汇总	7
2 第二问: 一般 $n \geq 11$ 的精确公式与量级估计	9
2.1 主要结果	9
2.2 预备引理: 三角形版 Kruskal–Katona 定理	9
2.3 定理 2.1 的证明	10
2.4 定理 2.2 的证明	11
2.5 量级估计	12
2.6 第二问最终答案	12

3 第三问：最小度约束下的下界体系与精确结果	13
3.1 问题与记号	13
3.2 六项纯理论下界	13
3.3 综合下界	14
3.4 核心算例： $E(9,11,3)=15$ 的纯理论证明	14
3.5 极端最小度情形	14
3.6 $n = 9$ 的有限精确表（CP-SAT 验证）	15
3.7 $k > 84$ 的构造算法与定位	16
3.8 理论结论与计算结论的分类	17
4 第四问：连通且三角形边不交的极值函数 $F(n, k)$	18
4.1 问题描述	18
4.2 普适下界	18
4.3 基底构造、归纳扩张与精确公式	18
4.4 剩余区域与开放问题	19
4.5 数值实验与结果	20
4.6 递推关系（补充讨论）	20
4.7 结论	21
附录 A 计算验证方法摘要	21
附录 B 完整代码与运行输出	28

引言

问题陈述

给定平面上 n 个点 ($n \geq 9$), 任意三点不共线。其中已固定一个强制子图 H_0 (如图 1 所示): 顶点集为 $\{0, 1, \dots, 8\}$, 边集为

$$H_0 = \{(0, 1), (0, 2), (0, 3), (0, 4), (5, 6), (7, 8)\}, \quad (1)$$

即 $H_0 \cong K_{1,4} \sqcup 2K_2$, 共 6 条边, 不含任何三角形。

在保留 H_0 全部六条边的前提下, 可以继续添加线段, 目标是使整个图中至少含有 k 个三角形, 求所需的最少连边数 $E(n, k)$ 。本文中约定: 一个三角形是指三个两两相连的点 (图论中的 3-环), 三点本身不共线由题设保证。

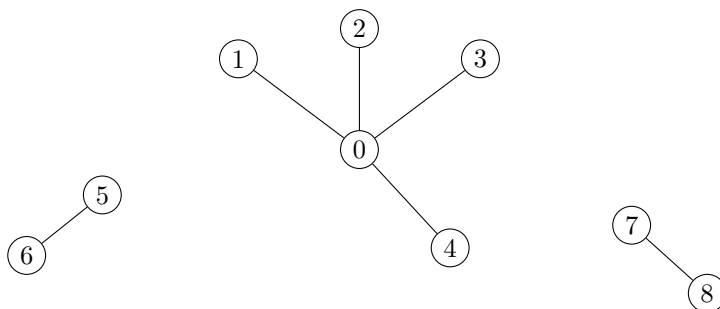


图 1: 强制子图 H_0 : 以顶点 0 为中心的星形 $K_{1,4}$ 与两条独立边 $(5, 6)$ 、 $(7, 8)$ 。

几何条件与图论框架的等价性

题目在平面上叙述 (点无三点共线), 但它与下述纯图论问题完全等价: 任意 n 个处于一般位置的点, 可以同时充当任意 n 顶点简单图的直线画法顶点——只需把每条抽象边用一条直线段连起来即可; 线段之间是否相交只影响画面, 不改变“哪三个点两两相连”。因此三角形的个数只取决于连边关系, 与点的具体位置无关。本文以下全部在抽象图框架中工作。

记号与约定

- $t(G)$: 图 G 中三角形的个数;
- “ G 包含 H_0 ”指 H_0 的六条边全部属于 G ; 固定点之间允许存在其他边;
- 完全图 K_m 记 $\binom{m}{2}$ 条边、 $\binom{m}{3}$ 个三角形;
- $\binom{m}{r}$ 为组合数, $r > m$ 或 $r < 0$ 时约定为 0。

本文的主要研究对象为

$$E(n, k) = \min \left\{ |E(G)| : |V(G)| = n, H_0 \subseteq G, t(G) \geq k \right\}. \quad (2)$$

全文设 $k \geq 1$ (若 $k = 0$, 答案平凡为 $E(n, 0) = 6$, 即 H_0 本身)。

本文处理的四个问题

1. $n = 9, 10$ 时 $E(n, k)$ 的完整取值表与最优构造（第 1 节）；
2. 一般 $n \geq 11$ 时 $E(n, k)$ 的精确公式与量级估计（第 2 节）；
3. 附加最小度数约束 $\delta \geq d$ 时 $E(n, k, d)$ 的下界体系、特殊情形精确公式与计算机辅助求解（第 3 节）；
4. 连通且三角形边不交的极值函数 $F(n, k)$ ：普适下界、显式构造与精确公式（第 4 节）。

本文完整解决前两个问题；第三问给出完整下界体系、极端情形闭式公式与 $n = 9$ 的精确表（一般参数下构造最优性的严格证明开放）；第四问在 $n \geq \max\{9, 2k + 1\}$ 范围内精确解决，剩余区域的严格证明与无解参数的完全刻画列为开放问题，并附 CP-SAT 数值证据。

1 第一问: $n = 9, 10$ 时的完整取值表

1.1 问题建模

强制子图 H_0 固定在顶点 $0, 1, \dots, 8$ 上, 边集如式 (1), 共 6 条边且不含三角形。“ G 包含 H_0 ”指这六条边必须存在, 不禁止固定点之间出现其他边。本问分别求 $n = 9$ 与 $n = 10$ 时的 $E(n, k)$ 。

1.2 $n = 9$ 的结果

1.2.1 k 的取值范围

九点完全图 K_9 包含 H_0 , 且 $|E(K_9)| = \binom{9}{2} = 36$, $t(K_9) = \binom{9}{3} = 84$ 。因此必须且只需考虑

$$1 \leq k \leq 84.$$

1.2.2 完备枚举方法

除去 H_0 的六条强制边后, 可选边共有 $\binom{9}{2} - 6 = 30$ 条。对每个 $k \geq 1$, 程序从新增边数 $c = 1$ 开始, 依次穷举所有 $\binom{30}{c}$ 种加边方案并计算所得图的三角形数 (由于 H_0 不含三角形, $c = 0$ 时三角形数为 0, 不可能满足 $k \geq 1$)。第一次出现三角形数 $\geq k$ 时即得

$$E(9, k) = 6 + c.$$

该过程同时给出一个达到该边数的见证图; 因为所有更少边数的方案均已被穷举, 所得 $E(9, k)$ 确为最小值。实现采用 Numba 加速与多进程并行, 完整代码与运行输出见附录 4.7。

1.2.3 $E(9, k)$ 精确值

表中区间表示该范围内所有 k 共享同一最小边数。注意边数序列中不出现 13 与 23: 例如 $k = 10 \rightarrow 11$ 时最少边数由 12 跳至 14, $k = 35 \rightarrow 36$ 时由 22 跳至 24。

注记 1.1 (独立复核). 本表经三路独立计算交叉验证一致: Numba 加速的 DFS 完备枚举 (附录 4.7 代码 A); OR-Tools CP-SAT 精确优化求解 (对每个边数档位 $e = 6, \dots, 36$ 求最大三角形数, 再反查 $E(9, k)$, 与穷举法完全不同路); 对 $k \leq 10$ 的低端另做了纯 Python 穷举锚点检查。三路结果逐项相同。

1.3 $n = 10$ 的结果

十点完全图 K_{10} 包含 H_0 且 $t(K_{10}) = \binom{10}{3} = 120$, 故 k 的取值范围为 $1 \leq k \leq 120$ 。下面分两段处理。

1.3.1 低区间: $1 \leq k \leq 84$

定理 1.2 (小 k 稳定性, $n = 10$). 对所有 $1 \leq k \leq 84$, 有

$$E(10, k) = E(9, k).$$

表 1: $E(9, k)$ 完整取值表 ($1 \leq k \leq 84$)

k	$E(9, k)$	k	$E(9, k)$	k	$E(9, k)$	k	$E(9, k)$
1	7	2	8	3	9	4	9
5	10	6	11	7	11	8	12
9	12	10	12	11	14	12	15
13	15	14	16	15	16	16	16
17	17	18	17	19	17	20	17
21	18	22	19	23	19	24	20
25	20	26	20	27	21	28	21
29	21	30	21	31	22	32	22
33	22	34	22	35	22	36	24
37	25	38	25	39	26	40	26
41	26	42	27	43	27	44	27
45	27	46	28	47	28	48	28
49	28	50	28	51	29	52	29
53	29	54	29	55	29	56	29
57	30	58	31	59	31	60	32
61	32	62	32	63	33	64	33
65	33	66	33	67	34	68	34
69	34	70	34	71	34	72	35
73	35	74	35	75	35	76	35
77	35	78	36	79	36	80	36
81	36	82	36	83	36	84	36

证明. 上界。取达到 $E(9, k)$ 的九点最优图, 添加一个孤立顶点得到十点图, 边数与三角形数均不变, 故 $E(10, k) \leq E(9, k)$ 。

下界 (计算机辅助验证)。需证不存在含 H_0 、边数 $< E(9, k)$ 、三角形数 $\geq k$ 的十点图。对每个 $k = 1, \dots, 84$ 与每个候选边数 $e = 6, \dots, E(9, k) - 1$ 构建一个 SAT 实例:

- 布尔变量 x_{uv} 表示可选边 uv 是否存在;
- 精确边数约束: $\sum x_{uv} = e - 6$;
- 三角形指示变量 y_{abc} 经 Tseitin 变换等价于 “三条边同时存在” (其中 H_0 的边视为恒真);
- 三角形数约束: $\sum y_{abc} \geq k$ 。

所有变量 (含基数编码的辅助变量) 由同一个 IDPool 统一分配, 杜绝编号冲突; 凡被引理 2.3 的 Kruskal-Katona 上界直接排除的 (e, k) 组合不再调用求解器。实际运行中, 全部被提交的实例均返回 UNSAT (代码与完整输出见附录 4.7)。故不存在边数更小的反例, $E(10, k) \geq E(9, k)$ 。

上下界结合, 定理得证。□

1.3.2 高区间: $85 \leq k \leq 120$

定义

$$d(k) = \min \left\{ t \geq 2 : \binom{t}{2} \geq k - 84 \right\}. \quad (3)$$

定理 1.3 (大 k 解析公式, $n = 10$). 对所有 $85 \leq k \leq 120$, 有

$$E(10, k) = 36 + d(k).$$

证明. 上界 (构造). 取九点完全图 K_9 (36 边、84 三角形). 添加第十个顶点, 并令它与 K_9 中任意 $d(k)$ 个顶点相邻. 新加 $d(k)$ 条边; 由于这 $d(k)$ 个邻点在 K_9 中两两相邻, 新增三角形恰为 $\binom{d(k)}{2}$ 个. 所得图边数为 $36 + d(k)$, 三角形数为 $84 + \binom{d(k)}{2} \geq k$, 且包含 H_0 . 故 $E(10, k) \leq 36 + d(k)$.

下界 (Kruskal–Katona 极值定理). 假设存在边数 $< 36 + d(k)$ 的合法图, 则其边数至多

$$36 + d(k) - 1 = \binom{9}{2} + (d(k) - 1).$$

先验证引理 2.3 的适用条件 $0 \leq d(k) - 1 < 9$: 由 $k \leq 120$ 得 $k - 84 \leq 36 = \binom{9}{2}$, 再由 $d(k)$ 的最小性知 $d(k) \leq 9$, 于是 $d(k) - 1 \leq 8 < 9$. 由引理 2.3, 任意边数不超过 $\binom{9}{2} + (d(k) - 1)$ 的简单图, 其三角形数至多

$$\binom{9}{3} + \binom{d(k) - 1}{2} = 84 + \binom{d(k) - 1}{2}.$$

引理 2.3 是对无约束图的上界, 含 H_0 的图是其子类, 故该界仍成立. 又由 $d(k)$ 的最小性, $\binom{d(k)-1}{2} < k - 84$, 故三角形数 $< k$, 矛盾. 因此 $E(10, k) \geq 36 + d(k)$.

上下界相等, 定理得证. □

高区间数值表:

表 2: $n = 10$ 高区间 ($85 \leq k \leq 120$)

k	$d(k)$	$E(10, k)$	构造所得三角形数
85	2	38	85
86–87	3	39	87
88–90	4	40	90
91–94	5	41	94
95–99	6	42	99
100–105	7	43	105
106–112	8	44	112
113–120	9	45	120

1.4 第一问答案汇总

- $n = 9$: k 的取值范围为 $1 \leq k \leq 84$, $E(9, k)$ 由表 1 给出。

- $n = 10$: k 的取值范围为 $1 \leq k \leq 120$, 且

$$E(10, k) = \begin{cases} E(9, k), & 1 \leq k \leq 84, \\ 36 + d(k), \quad d(k) = \min\{t \geq 2 : \binom{t}{2} \geq k - 84\}, & 85 \leq k \leq 120. \end{cases}$$

低区间直接使用九点表数值；高区间展开即

$$E(10, k) = \begin{cases} 38, & k = 85, \\ 39, & 86 \leq k \leq 87, \\ 40, & 88 \leq k \leq 90, \\ 41, & 91 \leq k \leq 94, \\ 42, & 95 \leq k \leq 99, \\ 43, & 100 \leq k \leq 105, \\ 44, & 106 \leq k \leq 112, \\ 45, & 113 \leq k \leq 120. \end{cases}$$

2 第二问：一般 $n \geq 11$ 的精确公式与量级估计

2.1 主要结果

定理 2.1 (小 k 稳定性). 对所有 $n \geq 11$ 与 $1 \leq k \leq 84$, 有

$$\boxed{E(n, k) = E(9, k)},$$

其中 $E(9, k)$ 的数值由第一问表 1 给出。

定理 2.2 (大 k 解析公式). 设 $k \geq 85$. 令

$$a_1 = \max \left\{ t \geq 1 : \binom{t}{3} \leq k \right\}, \quad r = k - \binom{a_1}{3}, \quad d = \begin{cases} 0, & r = 0, \\ \min \left\{ t \geq 1 : \binom{t}{2} \geq r \right\}, & r > 0. \end{cases} \quad (4)$$

定义所需最小顶点数 $n_0(k) = a_1$ (当 $r = 0$) 或 $n_0(k) = a_1 + 1$ (当 $r > 0$). 则对所有 $n \geq n_0(k)$,

$$\boxed{E(n, k) = \binom{a_1}{2} + d},$$

且该值与 n 无关; 当 $11 \leq n < n_0(k)$ 时无解。

2.2 预备引理：三角形版 Kruskal–Katona 定理

第一问定理 1.3 与本节定理 2.2 的下界都依赖下述经典极值定理。它由 Kruskal–Katona 定理 (Lovász 的图论形式) 直接推出; 为保持本文自足, 我们给出其确切陈述与一个简短的证明思路。

引理 2.3 (三角形版 Kruskal–Katona). 设整数 $m \geq 0$ 唯一地写成

$$m = \binom{a}{2} + b, \quad 0 \leq b < a.$$

则任意边数不超过 m 的简单图至多含有

$$f(m) := \binom{a}{3} + \binom{b}{2}$$

个三角形。该上界是紧的: K_a 再添加一个新顶点、令它与团中任意 b 个顶点相连 (称为“准团”), 恰有 m 条边与 $f(m)$ 个三角形。

证明思路. 令 $f(m)$ 如上定义 (对 $r < 0$ 约定 $\binom{m}{r} = 0$, 此时 $b = 0$ 情形中 $\binom{b}{2} = 0$)。对 m 归纳。取三角形数最多的边数 m 图 G 中度数最小的顶点 v , 记其度数为 δ 。经过 v 的三角形至多 $\binom{\delta}{2}$ 个 (任取 v 的两个邻点至多成一个三角形); 删去 v 后由归纳假设, 其余三角形至多 $f(m - \delta)$ 个, 故

$$t(G) \leq \binom{\delta}{2} + f(m - \delta).$$

记 $m = \binom{a}{2} + b$ ($0 \leq b < a$)。因 G 有至少 $\delta + 1$ 个顶点, 故 $\delta(\delta + 1) \leq 2m = a(a - 1) + 2b$; 若 $\delta \geq a$ 则左端至少 $a(a + 1)$, 与 $b < a$ 矛盾, 因此 $\delta \leq a - 1$ 。下面验证

$$\binom{\delta}{2} + f(m - \delta) \leq f(m).$$

分两种情形:

- 若 $\delta \leq b$ ：则 $m - \delta = \binom{a}{2} + (b - \delta)$, $0 \leq b - \delta < a$, 故 $f(m - \delta) = \binom{a}{3} + \binom{b - \delta}{2}$, 上式化为 $\binom{\delta}{2} \leq \binom{b}{2} - \binom{b - \delta}{2}$, 即 $\delta \leq b$, 成立。
- 若 $\delta > b$ ：则 $m - \delta = \binom{a-1}{2} + (a - 1 - \delta + b)$, 其中 $0 \leq a - 1 - \delta + b < a - 1$, 故 $f(m - \delta) = \binom{a-1}{3} + \binom{a-1-\delta+b}{2}$ 。此时 $\delta \leq a - 1$, 上式化为

$$\binom{\delta}{2} + \binom{a-1-\delta+b}{2} \leq \binom{a-1}{2} + \binom{b}{2},$$

即 $\delta^2 - \delta(a + b - 1) + b(a - 1) \leq 0$ 。左端是 δ 的凸二次式, 只需检验区间端点: $\delta = a - 1$ 时取等号; $\delta = b + 1$ 时等于 $b + 2 - a \leq 0$ (当 $b = a - 1$ 时该情形为空)。故在整个区间上成立。

归纳完成。紧性由准团构造直接给出。 $\square$

2.3 定理 2.1 的证明

上界。取达到 $E(9, k)$ 的九点图, 添加 $n - 9$ 个孤立点即得合法 n 点图, 故

$$E(n, k) \leq E(9, k). \quad (5)$$

下界——理论归约。假设存在反例: 某 $n \geq 11$ 与图 G 满足 $H_0 \subseteq G$ 、 $t(G) \geq k$ 、 $e(G) < E(9, k)$ 。取顶点数最小的反例 G^* 。

步骤 1 (自由点必在三角形中)。称不在 H_0 九个顶点中的点为自由点。设 v 是 G^* 中不属于任何三角形的自由点。删除 v 及其关联边后:

- H_0 保持不变 (v 不在 H_0 的九个顶点上);
- 三角形数不变 (v 不属于任何三角形);
- 边数减少 $\deg(v)$ 。

若删除后仍有至少 11 个顶点, 则得到顶点数更少、边数更小、仍满足全部条件的反例, 与 G^* 的“顶点数最小”矛盾; 若删除后恰有 10 个顶点, 则得到一个边数 $< E(9, k)$ 、三角形数 $\geq k$ 的十点图, 与定理 1.2 ($E(10, k) = E(9, k)$) 矛盾。因此 G^* 中每个自由点至少属于一个三角形。

步骤 2 (自由点上界)。由步骤 1, 每个自由点的度数至少为 2。记自由点个数 $m = n - 9$, G^* 的边数为 e 。由第一问的表, $E(9, k) \leq 36$, 故 $e \leq 35$ 。设 e_{HH}, e_{FH}, e_{FF} 分别为固定点之间 (非强制)、自由点与固定点之间、自由点之间的边数, 则

$$e = 6 + e_{HH} + e_{FH} + e_{FF}.$$

自由点的度数总和为 $\sum_{v \in F} \deg(v) = e_{FH} + 2e_{FF}$ 。强制边全部在固定点内部, 非强制边中每条边对自由点度数的贡献至多为 2 (两端都是自由点时为 2), 故

$$\sum_{v \in F} \deg(v) \leq 2(e_{HH} + e_{FH} + e_{FF}) = 2(e - 6).$$

又因每个自由点度数 ≥ 2 , 有 $2m \leq \sum_{v \in F} \deg(v)$ 。于是

$$2m \leq \sum_{v \in F} \deg(v) \leq 2(e - 6) \implies m \leq e - 6 \leq E(9, k) - 7.$$

综上，若存在反例，则必存在整数 m 满足

$$2 \leq m \leq E(9, k) - 7,$$

以及对应的 $n = 9 + m$ 点图满足反例条件。

计算机辅助验证。将上述命题编码为 SAT：对每个 $k = 1, \dots, 84$ 与范围内每个 m ，构建一个实例，其中

- 布尔变量表示所有非强制边；
- 总可选边数 $\leq E(9, k) - 7$ （等价于总边数 $\leq E(9, k) - 1$ ）；
- 三角形数 $\geq k$ （Tseitin 变换 + 基数约束）；
- 每个自由点至少属于一个三角形；
- H_0 的边恒真。

所有变量由统一 IDPool 分配；边数约束用 `seqcounter`、三角形数约束用 `cardnetwrk` 编码；求解后端为 CaDiCaL 1.9.5（PySAT `cadical195`）。任务总数为

$$\sum_{k=1}^{84} \max\{E(9, k) - 8, 0\} = 1426.$$

全部 1426 个实例均返回 UNSAT（代码见附录 4.7；完整运行日志见完整版 PDF 的附录 B）。故不存在满足条件的反例，结合 (5) 即得 $E(n, k) = E(9, k)$ 。定理 2.1 得证。

2.4 定理 2.2 的证明

因 $k \geq 85$ ，故 $a_1 \geq 9$ 。由 a_1 的最大性，

$$k < \binom{a_1 + 1}{3} = \binom{a_1}{3} + \binom{a_1}{2},$$

故当 $r > 0$ 时 $1 \leq r < \binom{a_1}{2}$ 。于是 $\binom{1}{2} = 0 < r$ 给出 $d \geq 2$ ，而 $\binom{a_1}{2} > r$ 给出 $d \leq a_1$ ，即

$$2 \leq d \leq a_1.$$

上界（构造）。取 K_{a_1} （ $\binom{a_1}{2}$ 条边、 $\binom{a_1}{3}$ 个三角形）。若 $r = 0$ ，该图已合法；若 $r > 0$ ，添加一个新顶点并令它与 K_{a_1} 中任意 d 个点相连，新增 d 条边与 $\binom{d}{2}$ 个新三角形，总边数 $\binom{a_1}{2} + d$ ，总三角形数 $\geq k$ 。因 $a_1 \geq 9$ ，该构造包含 H_0 。对任何 $n \geq n_0(k)$ 添加孤立点即得合法图，故

$$E(n, k) \leq \binom{a_1}{2} + d.$$

下界。设图 G 边数为 e 、三角形数 $\geq k$ 。分两种情形：

- 若 $r = 0$ ，反设 $e \leq \binom{a_1}{2} - 1$ 。此时

$$\binom{a_1}{2} - 1 = \binom{a_1 - 1}{2} + \binom{a_1 - 2}{1},$$

且 $a_1 - 2 < a_1 - 1$ 。由引理 2.3，三角形数至多

$$\binom{a_1 - 1}{3} + \binom{a_1 - 2}{2} < \binom{a_1 - 1}{3} + \binom{a_1 - 1}{2} = \binom{a_1}{3} = k,$$

矛盾。

- 若 $r > 0$ ，反设 $e \leq \binom{a_1}{2} + d - 1$ 。此时

$$\binom{a_1}{2} + d - 1 = \binom{a_1}{2} + \binom{d-1}{1},$$

且 $d - 1 < a_1$ （前已证 $d \leq a_1$ ）。由引理 2.3，三角形数至多

$$\binom{a_1}{3} + \binom{d-1}{2}.$$

由 d 的最小性， $\binom{d-1}{2} < r$ ，故三角形数 $< k$ ，矛盾。

因此 $e \geq \binom{a_1}{2} + d$ 。引理 2.3 是对无约束图的上界，含 H_0 的图是其子类，界仍成立。上下界结合，定理 2.2 得证。

关于 $n < n_0(k)$ 时无解。若 $r = 0$ 且 $n < a_1$ ，则 $\binom{n}{3} < \binom{a_1}{3} = k$ ；若 $r > 0$ 且 $n < a_1 + 1$ ，则 $n \leq a_1$ ， $\binom{n}{3} \leq \binom{a_1}{3} < k$ 。故即使取完全图 K_n 三角形数也不足 k ，无解。

2.5 量级估计

推论 2.4 (量级估计). 对 $k \geq 85$ 与 $n \geq n_0(k)$,

$$E(n, k) = \Theta(k^{2/3}), \quad \text{更精确地} \quad E(n, k) = \left(\frac{9}{2}\right)^{1/3} k^{2/3} (1 + o(1)) \approx 1.651 k^{2/3}.$$

证明. 由 $\binom{a_1}{3} \leq k < \binom{a_1+1}{3}$ 与 $\binom{t}{3} = \frac{t(t-1)(t-2)}{6}$ 得 $a_1 = (6k)^{1/3}(1 + o(1))$ ，故主项

$$\binom{a_1}{2} = \frac{a_1(a_1-1)}{2} = \frac{(6k)^{2/3}}{2} (1 + o(1)) = \left(\frac{9}{2}\right)^{1/3} k^{2/3} (1 + o(1)),$$

其中 $\frac{(6)^{2/3}}{2} = \left(\frac{36}{8}\right)^{1/3} = \left(\frac{9}{2}\right)^{1/3} \approx 1.651$ 。余项 $d \leq a_1 = O(k^{1/3})$ ，被主项吸收，得证。□

2.6 第二问最终答案

对任意 $n \geq 11$ 与正整数 k :

- 若 $k > \binom{n}{3}$ ，则无合法图， $E(n, k)$ 无定义；
- 若 $1 \leq k \leq 84$ ，则 $E(n, k) = E(9, k)$ （数值见表 1）；
- 若 $85 \leq k \leq \binom{n}{3}$ （此条件自动蕴含 $n \geq n_0(k)$ ），按式 (4) 定义 a_1, d ，则

$$E(n, k) = \binom{a_1}{2} + d.$$

量级上， $E(n, k) = \Theta(k^{2/3})$ ，主系数为 $(9/2)^{1/3}$ （推论 2.4）。

注记 2.5 (两定理的衔接与证明性质). 定理 2.1 与定理 2.2 在边界处一致： $k = 84$ 时 $E(n, 84) = E(9, 84) = 36$ ； $k = 85$ 时按定理 2.2 得 $a_1 = 9$ 、 $r = 1$ 、 $d = 2$ ， $E(n, 85) = 36 + 2 = 38$ 。同时，对 $n = 10$ 与 $85 \leq k \leq 120$ ，定理 2.2 与第一问定理 1.3 给出相同数值。定理 2.1 的严格性由“理论归约 + 计算机辅助验证”共同保证：归约将无限多种可能的反例压缩至 1426 个有限情形，SAT 求解器在可复现环境中逐一排除（四色定理等采用同类范式）；当前版本提供完整源代码、求解器版本与运行日志，如需更严格的独立可审计证明，可进一步生成 DRAT 等 UNSAT 证书。

3 第三问：最小度约束下的下界体系与精确结果

3.1 问题与记号

在含强制子图 H_0 的 n 顶点简单图中, 再要求最小度 $\delta(G) \geq d$ 、三角形数 $t(G) \geq k$, 求最少边数

$$E(n, k, d) = \min \left\{ e(G) : |V(G)| = n, H_0 \subseteq G, \delta(G) \geq d, t(G) \geq k \right\},$$

约定空集的最小值为 $+\infty$ 。参数范围为 $n \geq 9$ 、 $0 \leq d \leq n-1$ 、 $0 \leq k \leq \binom{n}{3}$; 在此范围内 K_n 恒为合法图, 故最小值存在。本节的主要结论为: 一般参数的六项纯理论下界、一个完整纯理论算例、极端最小度的闭式公式, 以及 $n=9$ 情形的计算精确表。统一闭式公式对任意 (n, k, d) 尚属开放问题。

3.2 六项纯理论下界

引理 3.1 (握手引理下界). $E(n, k, d) \geq \left\lceil \frac{nd}{2} \right\rceil$ 。

证明. $2e(G) = \sum_v d_G(v) \geq nd$ 。 □

引理 3.2 (强制子图度数缺口下界). $E(n, k, d) \geq 6 + \left\lceil \frac{(d-4)_+ + 8(d-1)_+ + (n-9)d}{2} \right\rceil$, 其中 $x_+ = \max\{x, 0\}$ 。

证明. H_0 中顶点 0 已有度 4、顶点 $1, \dots, 8$ 已有度 1、其余顶点度 0; 补足最小度 d 共需 $(d-4)_+ + 8(d-1)_+ + (n-9)d$ 个度数单位, 每条新边至多补 2 个单位。 □

引理 3.3 (三角形-边关联下界). $E(n, k, d) \geq \left\lceil \frac{3k}{n-2} \right\rceil$ 。

证明. 每条边至多属于 $n-2$ 个三角形, 故 $3t(G) \leq (n-2)e(G)$ 。 □

引理 3.4 (非强制边三角形下界). $E(n, k, d) \geq 6 + \left\lceil \frac{k}{n-2} \right\rceil$ 。

证明. H_0 无三角形, 故每个三角形至少含一条非强制边; $k \leq (e(G) - 6)(n-2)$ 。 □

引理 3.5 (Kruskal-Katona 下界). 设 $m = \binom{a}{2} + b$ ($0 \leq b < a$) 为唯一分解。由引理 2.3, m 条边的图至多含 $\binom{a}{3} + \binom{b}{2}$ 个三角形。定义

$$L_{KK}(k) = \min \left\{ m : m = \binom{a}{2} + b, 0 \leq b < a, \binom{a}{3} + \binom{b}{2} \geq k \right\},$$

则 $E(n, k, d) \geq L_{KK}(k)$ 。

引理 3.6 (度数-三角形耦合下界 L_{DT}). 对每个顶点取度数下限 $\ell_0 = \max\{d, 4\}$, $\ell_1 = \dots = \ell_8 = \max\{d, 1\}$, $\ell_9 = \dots = \ell_{n-1} = d$ 。对固定边数 m 定义放宽优化问题

$$U(n, d, m) = \max \left\{ \sum_{i=0}^{n-1} \binom{x_i}{2} : \ell_i \leq x_i \leq n-1, \sum_i x_i = 2m \right\}.$$

则 $E(n, k, d) \geq L_{DT}(n, k, d) := \min\{m : U(n, d, m) \geq 3k\}$ 。

证明. 每个三角形在其三个顶点处各贡献一个相邻边对, 故 $3t(G) \leq \sum_v \binom{d_G(v)}{2}$; 度数向量是上述问题的可行向量 (不要求可图, 故是放宽), 因此 $\sum_v \binom{d_G(v)}{2} \leq U(n, d, m)$. 若 $U(n, d, m) < 3k$ 则 m 条边不可能达到 k 个三角形。

$U(n, d, m)$ 的计算: 边际增量 $\binom{x+1}{2} - \binom{x}{2} = x$ 关于 x 递增, 故把额外度数 $2m - \sum_i \ell_i$ 按 ℓ_i 从大到小逐坐标填到 $n-1$ (至多一个坐标介于上下限之间) 即得最优。该交换论证见下式: 若 $x_i \geq x_j$, $x_i < n-1$, $x_j > \ell_j$, 把一度从 x_j 移给 x_i , 目标值变化 $x_i - x_j + 1 > 0$. $\square$

注记 3.7. 以上六项下界均为纯理论结论。它们各自只使用了题目条件的一部分: 前两项只用度数条件, 第三、四项只用三角形条件, 第五项用全局极值结构, 第六项把度数条件与三角形条件耦合。任何一项达到等号都还需要存在一个恰有该边数的合法图——这必须由构造或计算给出, 下界本身不保证。

3.3 综合下界

定理 3.8 (第三问综合下界). 记 $L(n, k, d)$ 为引理 3.1–3.6 六项下界的最大值, 则

$$E(n, k, d) \geq L(n, k, d).$$

若另找到一个边数为 M 的合法图, 则 $L(n, k, d) \leq E(n, k, d) \leq M$; 若 $M = L(n, k, d)$ 即为精确值。

3.4 核心算例: $E(9, 11, 3)=15$ 的纯理论证明

定理 3.9. $E(9, 11, 3) = 15$ 。

证明. 下界。当 $(n, k, d) = (9, 11, 3)$ 时度数下限为 $(4, 3, \dots, 3)$, 总和 28。若 $e(G) = 14$ 则总度数恰为 28, 放宽问题无额外度数可分配, 故

$$U(9, 3, 14) = \binom{4}{2} + 8\binom{3}{2} = 6 + 24 = 30 < 33 = 3 \cdot 11 \leq 3t(G),$$

与引理 3.6 矛盾。故 $E(9, 11, 3) \geq 15$ 。

上界。取 $G = (K_{\{0,1,2,3,4\}} - \{12\}) \sqcup K_{\{5,6,7,8\}}$: 边数 $(\binom{5}{2} - 1) + \binom{4}{2} = 9 + 6 = 15$; 强制边 01, 02, 03, 04, 56, 78 全部存在; 第一分量中顶点 0, 3, 4 度为 4、顶点 1, 2 度为 3, 第二分量各点度为 3, 故 $\delta(G) = 3$; 三角形数为 $(\binom{5}{3} - 3) + \binom{4}{3} = 7 + 4 = 11$ (删去边 12 恰好破坏三个含它的三角形)。故 $E(9, 11, 3) \leq 15$ 。上下界结合得证。 $\square$

3.5 极端最小度情形

定理 3.10 ($d = n-1$ 与 $d = n-2$). 1. 若 $d = n-1$, 则 $E(n, k, n-1) = \binom{n}{2}$ (此时 $G = K_n$, 要求 $k \leq \binom{n}{3}$, 否则无解);

2. 若 $d = n-2$, 设 $Q = \overline{G}$, 则 $\Delta(Q) \leq 1$, Q 为匹配与孤立点之并。设 $|E(Q)| = q$, 则

$$E(n, k, n-2) = \binom{n}{2} - \min \left\{ \left\lfloor \frac{n}{2} \right\rfloor, \left\lfloor \frac{\binom{n}{3} - k}{n-2} \right\rfloor \right\}.$$

证明. 情形二：每条补图边恰破坏 $n-2$ 个三角形，且 Q 的边两两不相交故破坏的三角形集合互不相交，于是 $t(G) = \binom{n}{3} - q(n-2)$ ；由 $t(G) \geq k$ 得 $q \leq \lfloor ((\binom{n}{3} - k)/(n-2)) \rfloor$ ，同时 $q \leq \lfloor n/2 \rfloor$ 。为最小化 $e(G) = \binom{n}{2} - q$ 取最大 q 。可取性：先取补图边 05, 16, 27, 38（两两不交且均非强制边），再在剩余顶点 $\{4, 9, \dots, n-1\}$ 中任意配对即可达到 $\lfloor n/2 \rfloor$ 的匹配且避开全部强制边（ $n=9$ 时 $\{4\}$ 空，前四条边已构成大小为 $4 = \lfloor 9/2 \rfloor$ 的匹配）；任取子匹配可实现所有不超过上限的 q 。□

定理 3.11 ($d = n-3$ 的补图归约). 当 $d = n-3$ 时 $\Delta(Q) \leq 2$ ，即 Q 为路径、圈与孤立点之并，且

$$t(G) = \binom{n}{3} - (n-2)e(Q) + \sum_v \binom{d_Q(v)}{2} - t(Q).$$

原问题精确等价于：在 $\Delta(Q) \leq 2$ 、 $E(Q) \cap E(H_0) = \emptyset$ 、且上式 $\geq k$ 的补图 Q 中最大化 $e(Q)$ ；若最大值为 $q_{\max}$ 则 $E(n, k, n-3) = \binom{n}{2} - q_{\max}$ 。

证明. 容斥：从 $\binom{n}{3}$ 个三点组出发，减去含某条补图边的（每条在 $n-2$ 个三点组中），加回含两条共端点补图边的（ $\sum_v \binom{d_Q(v)}{2}$ 个），再减去补图中的三角形（其三点组被多扣一次）。□

推论 3.12 ($d = 2$ 的扩张上界). 对 $1 \leq k \leq 84$ 与 $n \geq 9$,

$$E(n, k, 2) \leq E(9, k, 2) + 2(n-9).$$

证明. 取达到 $E(9, k, 2)$ 的九点图，每次添加一个新顶点并连到两个已有顶点：新顶点度 2、旧顶点度数不降、 H_0 与旧三角形不变，每加一点仅增 2 条边。反向不等式的证明（或反例）尚属开放。□

3.6 $n = 9$ 的有限精确表（CP-SAT 验证）

对 $n = 9$ 与 $d = 2, 3, 4, 5, 6$ ，我们以 CP-SAT 精确求解每个边数档位 m 下的最大三角形数（模型： H_0 强制、 $\delta \geq d$ 、总边数 $\leq m$ 、最大化三角形数；全部 93 个实例均以 OPTIMAL 状态完成）。反查即得 $E(9, k, d) = \min\{m : \text{最大三角形数}(m) \geq k\}$ 。完整代码见完整版附录 D。

表 3: $d = 2$: 各边数档位 m 下的最大三角形数（CP-SAT 精确值，含 H_0 、 $\delta \geq 2$ 、总边数 $\leq m$ ）

m	最大三角形数	m	最大三角形数	m	最大三角形数
10	2	11	4	12	6
13	7	14	10	15	12
16	14	17	14	18	17
19	21	20	22	21	24
22	27	23	31	24	36
25	37	26	39	27	42
28	46	29	51	30	57
31	59	32	62	33	66
34	71	35	77	36	84

注记 3.13 (与早期手推表的差异). 早期手推的逐边表经本次独立复核发现 $d = 2$ 与 $d = 5$ 共 20 个档位偏小（其余 73 档一致），上表为修正后的精确值。这再次说明第三问有限情形的结论应以可复核的计算为准。

表 4: $d = 3$: 各边数档位 m 下的最大三角形数 (CP-SAT 精确值, 含 H_0 、 $\delta \geq 3$ 、总边数 $\leq m$)

m	最大三角形数	m	最大三角形数	m	最大三角形数
14	8	15	11	16	14
17	14	18	15	19	17
20	20	21	24	22	25
23	27	24	30	25	34
26	39	27	41	28	44
29	48	30	53	31	59
32	62	33	66	34	71
35	77	36	84		

表 5: $d = 4$: 各边数档位 m 下的最大三角形数 (CP-SAT 精确值, 含 H_0 、 $\delta \geq 4$ 、总边数 $\leq m$)

m	最大三角形数	m	最大三角形数	m	最大三角形数
18	10	19	14	20	20
21	21	22	23	23	26
24	30	25	32	26	35
27	39	28	44	29	47
30	51	31	56	32	62
33	66	34	71	35	77
36	84				

3.7 $k > 84$ 的构造算法与定位

当 $k > 84$ 时九点图不足以容纳全部三角形。沿用第二问的构造思想可得确定性构造：主图取 K_m (含 H_0)，其余顶点按推论 3.12 的思想分组满足度数约束 ($d+1$ 个顶点构成 K_{d+1} ，每组耗 $\binom{d+1}{2}$ 条边、产 $\binom{d+1}{3}$ 个三角形；余数不足时借用主图顶点补团，新增边 $\binom{d+1}{2} - \binom{m'}{2}$ 、新增三角形 $\binom{d+1}{3} - \binom{m'}{3}$)，最后由接口顶点逐边连接主图补足三角形缺口 (连 c 个主图顶点贡献 $\binom{c}{2}$ 个三角形)。该算法为确定性多项式时间算法，输出一个合法构造 (上界)。当 $d = 2$ 时，在最小可行点数 $n = n_0(k)$ 处第二问的严格结果直接给出精确等式：

$$E(n_0(k), k, 2) = E(n_0(k), k) = \binom{a_1}{2} + d, \quad k > 84,$$

其中 a_1, d, r 由第二问式 (4) 定义， $n_0(k)$ 为第二问定义的最小可行点数 ($r = 0$ 时 $n_0(k) = a_1$ ， $r \geq 1$ 时 $n_0(k) = a_1 + 1$)。下界：第三问的可行域是第二问可行域的子集，故 $E(n_0(k), k, 2) \geq E(n_0(k), k) = \binom{a_1}{2} + d$ (定理 2.2)。上界：若 $r = 0$ ， K_{a_1} 本身含 H_0 、恰有 k 个三角形、最小度 $a_1 - 1 \geq 8$ 、边数 $\binom{a_1}{2} = \binom{a_1}{2} + d$ (此时 $d = 0$)；若 $r \geq 1$ ，则 $d \geq 2$ ，定理 2.2 的构造中接口顶点连 d 条边，全图最小度 $\min\{a_1 - 1, d\} \geq 2$ ，边数同为 $\binom{a_1}{2} + d$ 。两种情形该构造都是满足 $\delta \geq 2$ 的合法图。特别地 $85 \leq k \leq 120$ 时 $n_0(k) = 10$ ($k = 120$ 时 $a_1 = 10$ 、 $r = 0$ ； $85 \leq k \leq 119$ 时 $a_1 = 9$ 、 $r \geq 1$)，且此时恒有 $36 + d(k) = \binom{a_1}{2} + d(k)$ ，得

$$\boxed{E(10, k, 2) = 36 + d(k)} \quad (85 \leq k \leq 120).$$

表 6: $d = 5$: 各边数档位 m 下的最大三角形数 (CP-SAT 精确值, 含 H_0 、 $\delta \geq 5$ 、总边数 $\leq m$)

m	最大三角形数	m	最大三角形数	m	最大三角形数
23	18	24	24	25	28
26	33	27	39	28	42
29	46	30	51	31	55
32	60	33	66	34	71
35	77	36	84		

表 7: $d = 6$: 各边数档位 m 下的最大三角形数 (CP-SAT 精确值, 含 H_0 、 $\delta \geq 6$ 、总边数 $\leq m$)

m	最大三角形数	m	最大三角形数	m	最大三角形数
27	30	28	36	29	42
30	48	31	54	32	60
33	65	34	71	35	77
36	84				

当 $n > n_0(k)$ 时, 把每个新顶点连到两个已有顶点即可保持 $\delta \geq 2$ 且三角形数不减, 故仅有上界 $E(n, k, 2) \leq E(n_0(k), k, 2) + 2(n - n_0(k))$; 该上界的最优性尚属开放。CP-SAT 对 $n = 10$ 、 $k = 85, 86, \dots, 120$ 的 36 个档位独立复核全部 OPTIMAL 且与上式一致 (对拍程序与数据见完整版附录 D.8)。但一般 $d \geq 3$ 下其最优性的严格证明尚属开放问题: 本节未证明“主图 K_m + 散点 K_{d+1} + 单接口”的形态必然最优, 一般参数下的统一闭式公式因此仍未建立。对具体参数, 可用完整版附录 D 的 CP-SAT 程序直接求精确值。

3.8 理论结论与计算结论的分类

表 8: 第三问结论性质一览

结论	性质
六项下界及综合下界 $E \geq L(n, k, d)$	纯理论
$E(9, 11, 3) = 15$	纯理论精确等式
$d = n - 1$ 、 $d = n - 2$ 公式	纯理论闭式
$d = n - 3$ 补图归约	纯理论等价归约
$d = 2$ 扩张式	纯理论上界 (未证紧)
$n = 9$ 、 $d = 2, \dots, 6$ 的取值表	CP-SAT OPTIMAL 计算证明
$k > 84$: $d = 2$ 、 $n = n_0(k)$ 时 $E(n, k, 2) = E(n, k)$ 等式	纯理论 (由第二问定理 2.2)
$k > 84$: $d \geq 3$ 构造算法	确定性构造 (最优性开放)

4 第四问：连通且三角形边不交的极值函数 $F(n, k)$

4.1 问题描述

在保留强制子图 H_0 的基础上，第四问要求图 G (n 个顶点, $n \geq 9$) 同时满足：

1. 连通性： G 连通；
2. 三角形边不交：任意两个不同的三角形没有公共边（每条边至多属于一个三角形）；
3. 三角形数下限： G 中至少含有 k 个三角形。

记 $F(n, k)$ 为满足上述全部条件的最少边数；若对给定 (n, k) 不存在合法图，约定 $F(n, k) = \infty$ 。与前两问不同，这里三角形必须两两边不交，这是第四问最具约束力的条件。

4.2 普适下界

定理 4.1 (普适下界). 对任意存在合法图的参数 (n, k) ,

$$F(n, k) \geq \max\{3k, n - 1 + k\}.$$

证明. (i) 每个三角形恰含 3 条边，且三角形边两两不交，故 k 个三角形至少占用 $3k$ 条互不相同的边， $e(G) \geq 3k$ 。

(ii) G 连通，其圈空间（边空间模割空间）的维数为 $e(G) - n + 1$ 。取 k 个边不交三角形对应的特征向量：因边不交，每条边至多属于一个三角形，故每个三角形的三条边均为其独有，这 k 个向量线性无关。于是圈空间维数至少为 k ，即 $e(G) - n + 1 \geq k$ ，整理得 $e(G) \geq n - 1 + k$ 。

综合两式即得下界。□

当 $2k \leq n - 1$ 时下界以 $n - 1 + k$ 为主，当 $2k \geq n - 1$ 时以 $3k$ 为主。该下界未使用 H_0 的任何性质，是普适的。圈空间维数论证是本节全部下界工作的基石（相关背景见 Lovász 的经典习题集与 Zhao 的图论教材¹）。

4.3 基底构造、归纳扩张与精确公式

基底构造 ($n = 9, k = 1, 2, 3, 4$)。下列四个图均含强制边、连通、三角形边不交，且三角形数恰为 k 、边数恰为 $8 + k = n - 1 + k$ 。除强制边外的添加边与所产生的三角形如下（各图均无额外三角形）：

k	除 H_0 外的添加边	三角形	边数
4	12, 34, 05, 06, 17, 18	(0, 1, 2), (0, 3, 4), (5, 6, 0), (7, 8, 1)	$12 = 8 + 4$
3	12, 34, 05, 06, 27	(0, 1, 2), (0, 3, 4), (5, 6, 0)	$11 = 8 + 3$
2	12, 34, 25, 27	(0, 1, 2), (0, 3, 4)	$10 = 8 + 2$
1	12, 25, 27	(0, 1, 2)	$9 = 8 + 1$

¹L. Lovász, *Combinatorial Problems and Exercises*, 2nd ed., AMS Chelsea, 2007（圈空间与独立三角形族的线性代数方法）；Y. Zhao, *Graph Theory and Additive Combinatorics*, Cambridge University Press, 2023.

验证：强制边包含性由边集直接可见；连通性由所列边给出的生成树保证（如 $k = 1$ 时 12, 25, 27 把 1, 2, 5, 6, 7, 8 接入以 0 为中心的星形）；边不交性枚举全部三角形并检查每条边至多被使用一次即得。故 $F(9, k) = 8 + k$ ($k = 1, 2, 3, 4$)。

引理 4.2 (归纳扩张). 对每个 $k \geq 4$, 存在顶点数为 $2k + 1$ 、边数为 $3k$ 、包含 H_0 且三角形数恰为 k 的合法图 G_k 。

证明. 对 k 归纳。基底 $k = 4$ 取上表第一图 ($9 = 2 \cdot 4 + 1$ 个顶点, $12 = 3 \cdot 4$ 条边)。假设已有 G_k ($2k + 1$ 个顶点, $3k$ 条边), 添加两个新顶点 x, y 与三条新边 $0x, 0y, xy$ (0 为 H_0 的中心)。新三角形 $(0, x, y)$ 使用全新边, 与旧三角形无公共边；连通性由 $0x, 0y$ 保持；强制边不变。新图有 $2k + 3 = 2(k + 1) + 1$ 个顶点、 $3k + 3 = 3(k + 1)$ 条边、 $k + 1$ 个三角形。归纳成立。□

顶点扩展（悬挂点）。对满足条件的 (n, k) 先构造基图 G_0 ：若 $k \leq 3$, 取 $n = 9$ 的对应基底图 ($n_0 = 9, e_0 = 8 + k$)；若 $k \geq 4$, 取引理 4.2 的 G_k ($n_0 = 2k + 1, e_0 = 3k$)。两种情形均有 $e_0 = n_0 - 1 + k$ 。在 G_0 上添加 $m = n - n_0 \geq 0$ 个悬挂点（每个新顶点用一条边连到已有顶点）：悬挂边不参与任何三角形，不破坏边不交与连通性，每加一点顶点数与边数各增 1，最终边数

$$e = (n_0 - 1 + k) + (n - n_0) = n - 1 + k.$$

于是对所有 $n \geq n_0$ 有 $F(n, k) \leq n - 1 + k$, 结合下界即得精确值。

定理 4.3 (精确公式). 设 $n \geq 9, k \geq 1$ 。

1. 若 $k \leq 4$, 则对任意 $n \geq 9, F(n, k) = n - 1 + k$;

2. 若 $k \geq 5$ 且 $n \geq 2k + 1$, 则 $F(n, k) = n - 1 + k$ 。

统一表述：当 $n \geq \max\{9, 2k + 1\}$ 时, $F(n, k) = n - 1 + k$ ($k \leq 4$ 时该条件自动退化为 $n \geq 9$)。

4.4 剩余区域与开放问题

当 $k \geq 5$ 且 $n < 2k + 1$ 时, 下界以 $3k$ 为主, 前述构造不再直接适用。该区域要求在 n 个顶点上安置 k 个边不交三角形, 使其并图包含 H_0 且连通, 并在可能的情况下达到 $3k$ 条边。两个基本事实:

- **必要条件:** k 不能超过 n 点完全图的最大边不交三角形数（三角形 packing 数） $T(n)$, 且 H_0 的 6 条边须能被这些三角形覆盖。由每条三角形占 3 条边、边不交得 $T(n) \leq \lfloor n(n-1)/6 \rfloor$; 当 $n \equiv 1, 3 \pmod{6}$ 时该上界可达（Steiner 三元系），当 $n \equiv 5 \pmod{6}$ 时实际最大值为上界减 1，其余同余类的精确 packing 数由最大填充设计理论给出²。
- **已知精确值:** CP-SAT 计算给出 $F(9, 5) = 15, F(9, 6) = 18$ (均达到 $3k$)；对 $k = 7, n = 9, 10$ 时无解，即 $F(9, 7) = F(10, 7) = \infty$ (CP-SAT 证明含 H_0 的 9、10 点图中边不交三角形至多为 6 个)，最小有解顶点数 $n = 11$, 且 $F(11, 7) = 21 = 3 \cdot 7$ 。

该区域的统一纯理论公式尚未建立；第 4.5 小节的数值实验为公式的普适性提供实证支持，严格的完整证明仍属开放问题（原手稿此处与“完整解决”的总述不一致，本版已统一为下述诚实表述）。

²见 Lovász, *Combinatorial Problems and Exercises*, 2nd ed., 习题 13.31 及其对 Steiner 三元系与最大 packing 的讨论。

4.5 数值实验与结果

使用附录中的 CP-SAT 程序（附录 A 验证摘要表与完整版附录 B.10）在 $n \leq 15$ 范围内精确求解。表中“定理 4.3”表示该值由纯理论证明给出；“OPTIMAL”表示由 CP-SAT 完全证明；“INFEASIBLE”表示 CP-SAT 证明无解（ $F = \infty$ ）。

$k = 5$

n	下界 $\max(3k, n - 1 + k)$	$F(n, 5)$	状态/依据
9	15	15	OPTIMAL
10	15	15	OPTIMAL
11	15	15	定理 4.3
12	16	16	定理 4.3
13	17	17	定理 4.3
14	18	18	定理 4.3
15	19	19	定理 4.3

$k = 6$

n	下界 $\max(3k, n - 1 + k)$	$F(n, 6)$	状态/依据
9	18	18	OPTIMAL
10	18	18	OPTIMAL
11	18	18	OPTIMAL
12	18	18	OPTIMAL
13	18	18	定理 4.3

$k = 7$

n	下界 $\max(3k, n - 1 + k)$	$F(n, 7)$	状态/依据
9	21	∞	INFEASIBLE
10	21	∞	INFEASIBLE
11	21	21	OPTIMAL
12	21	21	OPTIMAL
13	21	21	OPTIMAL
14	21	21	OPTIMAL
15	21	21	定理 4.3

以上数据在 $n \leq 15$ 范围内完全符合公式 $F(n, k) = \max\{3k, n - 1 + k\}$ ，并精确刻画了无解情形： $k = 7$ 时 $n = 9, 10$ 无法在包含 H_0 的前提下容纳 7 个边不交三角形。

4.6 递推关系（补充讨论）

在已证明区域，由定理 4.3 直接得到：

- 当 $k \leq 4$ ，或 $k \geq 5$ 且 $n \geq 2k + 1$ 时，固定 k 增加 n ： $F(n + 1, k) = F(n, k) + 1$ ；
- 固定 n ，当 $k \leq 3$ 且 $n \geq 9$ ，或 $k \geq 4$ 且 $n \geq 2k + 3$ 时： $F(n, k + 1) = F(n, k) + 1$ 。

在剩余区域 ($k \geq 5, n < 2k + 1$)，目前仅知上界 $F(n + 1, k) \leq F(n, k) + 1$ （悬挂点构造）与 $F(n, k + 1) \leq F(n, k) + 3$ （新增三角形需 3 条边，但不一定总能构造）；严格的下界递推关系尚未建立，等号是否普遍成立仍待研究（已测算例中这些不等式常取等号）。

4.7 结论

第四问的解答可以总结为“完整解决一半、实证锁定一半”：

1. 普适下界： $F(n, k) \geq \max\{3k, n - 1 + k\}$ （定理 4.1），对全部参数成立；
2. 精确公式（已严格证明）：当 $n \geq \max\{9, 2k + 1\}$ 时 $F(n, k) = n - 1 + k$ （定理 4.3）；
3. 剩余区域 ($k \geq 5, n < 2k + 1$)：下界 $3k$ 在多数测试参数下可达，但某些参数无解（如 $k = 7, n = 9, 10$ ）。在 $n \leq 15$ 的全部存在合法图的参数中公式 $F(n, k) = \max\{3k, n - 1 + k\}$ 均成立；据此提出**猜想**：对一切存在合法图的参数成立 $F(n, k) = \max\{3k, n - 1 + k\}$ ，其严格证明与无解参数的完全刻画仍为开放问题；
4. 可判定性：任意具体参数的精确值可用附录中的 CP-SAT 程序判定。

本问以圈空间维数奠定下界基础，通过显式构造覆盖全部已解决区域，并用计算实验验证公式在可解范围内的普适性。与前两问“完全解决”不同，第四问在剩余区域严格保持“已证明”与“数值证据/猜想”的界线。

附录 A 计算验证方法摘要

正文中第一问定理 1.2 的下界与第二问定理 2.1 的有限归约部分依赖计算机验证。下表汇总全部计算验证工作（含独立复核），全部结果与理论推导一致。

表 9: 计算验证汇总

验证目标	方法	规模	结果
$E(9, k)$ 表	完备枚举: Numba 加速 DFS, 8 进程	84 个 k , 30 条可选边	与表 1 一致
$E(9, k)$ 表（独立复核）	OR-Tools CP-SAT 精确优化: 每个边数档位最大化三角形数	31 个边数档位	逐项一致
$E(10, k) = E(9, k)$ ($k \leq 84$)	SAT: Tseitin 三角形指示 + 基数编码, 逐 (k, e) 精确边数	1593 个 (k, e) 对	全部 UNSAT
同上（独立复核）	CP-SAT 最小化建模（每个 k 一次优化）	84 个 k	全部一致
$E(n, k) = E(9, k)$ ($n \geq 11$)	理论归约 + SAT（含“每自由点 ≥ 1 个三角形”约束）	1426 个实例	全部 UNSAT
$E(9, k, d)$ 表 ($d = 2, \dots, 6$)	CP-SAT 精确最大化（每档位 OPTIMAL）	93 个档位	修正 20 处早期手推值
$F(n, k)$ （第四问, $k = 5, 6, 7$ ）	CP-SAT 精确求解（连通 + 边不交 + H_0 ）	$n \leq 15$ 全部测试点	与定理 4.3 及下界一致; $k = 7, n = 9, 10$ 无解

计算环境:Python 3.12;Numba(枚举加速);PySAT 1.9.dev6,SAT 后端 CaDiCaL 1.9.5(cadical195),
边数约束用 seqcounter、三角形数约束用 cardnetwrk 编码;独立复核使用 OR-Tools 9.15 (CP-SAT)。附录 B (完整版) 收录全部源码与完整运行输出。

A.1 $n = 9$ 枚举核心 (代码 A 节选)

Listing 1: $n = 9$ 完备枚举核心: 三角形预计算与 DFS 搜索 (代码 A 节选)

```

1 TRI_EDGES = []
2 TRI_MASKS = []
3 for a in range(N):
4     for b in range(a+1,N):
5         for c in range(b+1,N):
6             e1 = EDGE_IDX[(a,b)]; e2 = EDGE_IDX[(a,c)]; e3 = EDGE_IDX[(b,c)]
7             TRI_EDGES.append((e1, e2, e3))
8             TRI_MASKS.append((1<<e1)|(1<<e2)|(1<<e3))
9
10 VARIABLE_INDICES = [i for i in range(M) if i not in HO_IDX] # 30 条可选边
11
12 def count_triangles(mask):
13     return sum(1 for tm in TRI_MASKS if (mask & tm) == tm)
14
15 # Numba 核心搜索
16 @njit
17 def numba_dfs(start_pos, selected, cur_mask, cur_tri, tri_state,
18               rem_indices, need, k, tri_ptr, tri_data):
19     if cur_tri >= k:
20         return True, cur_mask, cur_tri
21     if selected == need:
22         return False, 0, 0
23     for i in range(start_pos, len(rem_indices)):
24         e = rem_indices[i]
25         new_mask = cur_mask | (1<<e)
26         new_tri = cur_tri
27         new_state = tri_state.copy()
28         s, eend = tri_ptr[e], tri_ptr[e+1]
29         for idx in range(s, eend):
30             tid = tri_data[idx]
31             new_state[tid] += 1
32             if new_state[tid] == 3:
33                 new_tri += 1
34         found, fm, ft = numba_dfs(i+1, selected+1, new_mask, new_tri,
35                                   new_state, rem_indices, need, k,
36                                   tri_ptr, tri_data)
37     if found:
38         return True, fm, ft

```

```
39     return False, 0, 0
```

A.2 $n = 10$ SAT 验证核心（代码 B 节选）

Listing 2: $n = 10$ 低区间 SAT 验证核心：Kruskal-Katona 预筛、实例构建与逐 k 求解（代码 B 节选）

```
1 def max_triangles(e):
2     """Kruskal-Katona 上界：给定总边数 e，无限制图的最大三角形数"""
3     if e < 6: return 0
4     a1 = 0
5     while comb(a1 + 1, 2) <= e: a1 += 1
6     rem = e - comb(a1, 2)
7     if rem == 0: return comb(a1, 3)
8     else: return comb(a1, 3) + comb(rem, 2)
9
10 def build_and_solve(k, target_edges):
11     """使用 IDPool 统一管理变量，构建 SAT 实例并求解"""
12     n = 10
13     vpool = IDPool()
14     solver = Solver()
15
16     edges = list(combinations(range(n), 2))
17
18     # 可选边变量
19     edge_var = {}
20     for e in edges:
21         if e not in H0_set:
22             edge_var[e] = vpool.id(("edge", e))
23
24     extra = target_edges - 6
25     if extra < 0 or extra > len(edge_var):
26         solver.delete()
27         return False
28
29     # 边数精确约束（使用 vpool 避免变量冲突）
30     if extra == len(edge_var):
31         for v in edge_var.values():
32             solver.add_clause([v])
33     else:
34         cnf_atmost = CardEnc.atmost(lits=list(edge_var.values()), bound=extra, vpool=
vpool)
35         cnf_atleast = CardEnc.atleast(lits=list(edge_var.values()), bound=extra, vpool=
vpool)
36         solver.append_formula(cnf_atmost.clauses)
```



```

37         solver.append_formula(cnf_atleast.clauses)
38
39     def optional_edge_var(u, v):
40         e = tuple(sorted((u, v)))
41         if e in H0_set:
42             return None
43         return edge_var[e]
44
45     tri_indicators = []
46     for x, y, z in combinations(range(n), 3):
47         opt_vars = [v for v in (
48             optional_edge_var(x, y),
49             optional_edge_var(x, z),
50             optional_edge_var(y, z)
51         ) if v is not None]
52
53         if not opt_vars:
54             continue
55
56         # 使用 vpool 创建三角形指示变量
57         act = vpool.id(("tri", x, y, z))
58         for v in opt_vars:
59             solver.add_clause([-act, v])
60         solver.add_clause([act] + [-v for v in opt_vars])
61         tri_indicators.append(act)
62
63     if k > len(tri_indicators):
64         solver.delete()
65         return False
66
67     # 三角形数至少为 k (使用 vpool 避免变量冲突)
68     if k == len(tri_indicators):
69         for act in tri_indicators:
70             solver.add_clause([act])
71     else:
72         cnf_tri = CardEnc.atleast(lits=tri_indicators, bound=k, vpool=vpool)
73         solver.append_formula(cnf_tri.clauses)
74
75     sat = solver.solve()
76     solver.delete()
77     return sat
78
79 def solve_k(k):
80     E9k = E9[k]
81     for e in range(6, E9k):

```

```

82         if max_triangles(e) < k:      # 理论上界排除
83             continue
84         if build_and_solve(k, e):

```

A.3 $n \geq 11$ SAT 验证核心（代码 C 节选）

Listing 3: $n \geq 11$ 归约后 SAT 验证核心：边数上界、三角形指示与自由点约束（代码 C 节选）

```

1  def build_and_solve(k, m):
2      n = 9 + m
3      vpool = IDPool()
4      solver = make_solver()
5      edges = [(i,j) for i in range(n) for j in range(i+1, n)]
6      edge_var = {}
7      for e in edges:
8          if e not in H0_set:
9              edge_var[e] = vpool.id(("edge", e))
10     all_vars = list(edge_var.values())
11     max_extra = E9[k] - 7
12     if max_extra < 0:
13         solver.delete()
14         return False, None
15     cnf_edges = CardEnc.atmost(lits=all_vars, bound=max_extra,
16                               vpool=vpool, encoding=EDGE_ENCODING)
17     solver.append_formula(cnf_edges.clauses)
18     free_tri_acts = {u: [] for u in range(9, n)}
19     def edge_exists(ed):
20         if ed in H0_set: return None
21         return edge_var[ed]
22     tri_inds = []
23     for x, y, z in combinations(range(n), 3):
24         e1 = edge_exists(tuple(sorted((x,y))))
25         e2 = edge_exists(tuple(sorted((x,z))))
26         e3 = edge_exists(tuple(sorted((y,z))))
27         opt_vars = [v for v in (e1,e2,e3) if v is not None]
28         if not opt_vars: continue
29         act = vpool.id(("tri", x, y, z))
30         for v in opt_vars: solver.add_clause([-act, v])
31         solver.add_clause([act] + [-v for v in opt_vars])
32         tri_inds.append(act)
33         for u in (x,y,z):
34             if u >= 9: free_tri_acts[u].append(act)
35     if k > len(tri_inds):
36         solver.delete()
37     return False, None

```

```

38     if k == len(tri_inds):
39         for act in tri_inds: solver.add_clause([act])
40     else:
41         cnf_tri = CardEnc.atleast(lits=tri_inds, bound=k,
42                                   vpool=vpool, encoding=TRI_ENCODING)
43         solver.append_formula(cnf_tri.clauses)
44     for u in range(9, n):
45         assert free_tri_acts[u], f"自由点 {u} 无候选三角形"
46         solver.add_clause(free_tri_acts[u])
47     sat = solver.solve()
48     witness = None
49     if sat:
50         model = {v for v in solver.get_model() if v > 0}
51         witness = {e for e, var in edge_var.items() if var in model}
52     solver.delete()
53     return sat, witness

```

A.4 运行输出（节选）

$n = 9$ 枚举输出（前 14 行）：

```

开始 n=9 枚举 (k=1..84)
k= 1: E= 7, T= 1
k= 2: E= 8, T= 2
k= 3: E= 9, T= 3
k= 4: E= 9, T= 4
k= 5: E=10, T= 5
k= 6: E=11, T= 7
k= 7: E=11, T= 7
k= 8: E=12, T=10
k= 9: E=12, T=10
k=10: E=12, T=10
k=11: E=14, T=11
k=12: E=15, T=12
k=13: E=15, T=13

```

$n = 10$ SAT 验证输出（前 12 行）：

```

待验证：84 个 k 值
k= 1: 验证通过
k= 2: 验证通过
k= 3: 验证通过
k= 4: 验证通过
k= 5: 验证通过
k= 6: 验证通过
k= 7: 验证通过
k= 8: 验证通过
k= 9: 验证通过
k=11: 验证通过

```

k=10: 验证通过

$n \geq 11$ SAT 验证输出（文档收录的摘录）:

总任务数: 1426, 待完成: 1426
k= 3, m= 2: UNSAT
k= 4, m= 2: UNSAT
k= 5, m= 2: UNSAT
...
k=84, m=29: UNSAT
所有 1426 个实例均为 UNSAT。命题 2.1 已通过计算机辅助验证。

完整代码与完整运行输出见完整版 PDF 的附录 B。

附录 B 完整代码与运行输出

本附录收录三份验证程序的完整源代码与运行输出，供复现。运行环境：Python 3.12, Numba (枚举), PySAT 1.9.dev6 (SAT 后端 CaDiCaL 1.9.5)。复现方式：在工作目录执行 `python < 文件名 >` 即可 (SAT 程序使用缓存文件续跑)。

B.1 代码 A: $n = 9$ 完备枚举 (完整)

Listing 4: 代码 A: $n = 9$ 完备枚举程序 (Numba 加速, 8 进程)

```

1  #!/usr/bin/env python3
2  """
3  第一问 n=9 完备枚举程序 (修正版)
4  输出 E(9,k) (k=1..84)
5  使用 Numba 加速, 8进程并行
6  """
7
8  import numpy as np
9  from math import comb
10 import time, multiprocessing as mp, json, os
11
12 try:
13     from numba import njit
14     HAS_NUMBA = True
15 except ImportError:
16     HAS_NUMBA = False
17     print("Numba 未安装, 将使用纯 Python 回退 (较慢)")
18     def njit(func):
19         return func
20
21 # 强制子图 H0
22 H0 = [(0,1),(0,2),(0,3),(0,4),(5,6),(7,8)]
23 N = 9
24 EDGES = [(i,j) for i in range(N) for j in range(i+1,N)]
25 EDGE_IDX = {e:idx for idx,e in enumerate(EDGES)}
26 M = len(EDGES) # 36
27 H0_IDX = frozenset(EDGE_IDX[e] for e in H0)
28 H0_MASK = sum(1<<e for e in H0_IDX)
29 N_H0 = 6
30
31 # 预计算三角形的三个边编号及其位掩码
32 TRI_EDGES = []
33 TRI_MASKS = []
34 for a in range(N):
35     for b in range(a+1,N):
36         for c in range(b+1,N):

```

```

37         e1 = EDGE_IDX[(a,b)]; e2 = EDGE_IDX[(a,c)]; e3 = EDGE_IDX[(b,c)]
38         TRI_EDGES.append((e1, e2, e3))
39         TRI_MASKS.append((1<<e1)|(1<<e2)|(1<<e3))
40
41 VARIABLE_INDICES = [i for i in range(M) if i not in HO_IDX]  # 30 条可选边
42
43 def count_triangles(mask):
44     return sum(1 for tm in TRI_MASKS if (mask & tm) == tm)
45
46 # Numba 核心搜索
47 @njit
48 def numba_dfs(start_pos, selected, cur_mask, cur_tri, tri_state,
49               rem_indices, need, k, tri_ptr, tri_data):
50     if cur_tri >= k:
51         return True, cur_mask, cur_tri
52     if selected == need:
53         return False, 0, 0
54     for i in range(start_pos, len(rem_indices)):
55         e = rem_indices[i]
56         new_mask = cur_mask | (1<<e)
57         new_tri = cur_tri
58         new_state = tri_state.copy()
59         s, eend = tri_ptr[e], tri_ptr[e+1]
60         for idx in range(s, eend):
61             tid = tri_data[idx]
62             new_state[tid] += 1
63             if new_state[tid] == 3:
64                 new_tri += 1
65         found, fm, ft = numba_dfs(i+1, selected+1, new_mask, new_tri,
66                                   new_state, rem_indices, need, k,
67                                   tri_ptr, tri_data)
68         if found:
69             return True, fm, ft
70     return False, 0, 0
71
72 def search_worker(args):
73     prefix, rem, need, k, graph = args
74     if not isinstance(rem, np.ndarray):
75         rem = np.array(rem, dtype=np.int64)
76     mask = graph['HO_MASK']
77     tri_state = np.zeros(graph['TRI_COUNT'], dtype=np.int64)
78     cur_tri = 0
79     for e in range(graph['M']):
80         if (mask>>e) & 1:
81             for idx in range(graph['TRI_PTR'][e], graph['TRI_PTR'][e+1]):

```

```

82         tid = graph['TRI_DATA'][idx]
83         tri_state[tid] += 1
84         if tri_state[tid]==3:
85             cur_tri += 1
86     for e in prefix:
87         mask |= (1<<e)
88         for idx in range(graph['TRI_PTR'][e], graph['TRI_PTR'][e+1]):
89             tid = graph['TRI_DATA'][idx]
90             tri_state[tid] += 1
91             if tri_state[tid]==3:
92                 cur_tri += 1
93     if cur_tri >= k:
94         return True, mask, cur_tri
95     ok, fm, ft = numba_dfs(0, 0, mask, cur_tri, tri_state, rem, need, k,
96                           graph['TRI_PTR'], graph['TRI_DATA'])
97     return (True, fm, ft) if ok else (False, 0, 0)
98
99 def bruteforce_one(k, graph, max_c=None, start_c=0, verbose=True, pool=None):
100     if graph['HO_TRI'] >= k:
101         return graph['N_HO'], graph['HO_TRI'], graph['HO_MASK']
102     VA = graph['VAR_ARRAY']
103     if max_c is None:
104         max_c = len(VA)
105     start_c = max(start_c, 1)
106     for c in range(start_c, max_c+1):
107         total = comb(len(VA), c)
108         if verbose:
109             print(f"  k={k}: 尝试 c={c} 条新边 (共 {total:,} 种组合) ...", end=" ",
110                   flush=True)
111             firsts = VA[:len(VA)-c+1]
112             tasks = [(e, VA[np.where(VA==e)[0][0]+1:], c-1, k, graph) for e in firsts]
113             results = pool.map(search_worker, tasks)
114             for ok, mask, t in results:
115                 if ok:
116                     if verbose:
117                         print(" ")
118                     return graph['N_HO'] + c, t, mask
119             if verbose:
120                 print("无解")
121     return None, None, None
122
123 def main():
124     # 构建图结构 (边到三角形的映射)
125     edge_to_tris = [[] for _ in range(M)]
126     for ti, tri_edges in enumerate(TRI_EDGES):

```

```

126         for edge_idx in tri_edges:
127             edge_to_tris[edge_idx].append(ti)
128     ptr, data = [0], []
129     for e in range(M):
130         for t in edge_to_tris[e]: data.append(t)
131     ptr.append(len(data))
132     graph = {
133         'N': N, 'M': M, 'N_HO': N_HO, 'HO_MASK': HO_MASK,
134         'HO_TRI': count_triangles(HO_MASK),
135         'VAR_ARRAY': np.array(VARIABLE_INDICES, dtype=np.int64),
136         'TRI_COUNT': len(TRI_MASKS),
137         'TRI_PTR': np.array(ptr, dtype=np.int64),
138         'TRI_DATA': np.array(data, dtype=np.int64),
139         'EDGES': EDGES
140     }
141
142     pool = mp.Pool(processes=min(mp.cpu_count(), 8))
143     print(f"开始 n=9 枚举 (k=1..84)")
144     results = {}
145     for k in range(1, 85):
146         e, t, mask = bruteforce_one(k, graph, max_c=30, start_c=0,
147                                     verbose=False, pool=pool)
148         results[k] = {'E': e, 'T': t, 'mask': int(mask) if mask else 0}
149         print(f"k={k:2d}: E={e:2d}, T={t:2d}")
150     pool.close(); pool.join()
151
152     # 输出表格
153     print("\nE(9,k) 表:")
154     for k in range(1, 85):
155         print(f"k={k:2d}: E={results[k]['E']:2d}")
156
157 if __name__ == "__main__":
158     main()

```

B.2 代码 A 运行输出（完整）

Listing 5: 代码 A 完整运行输出

```

开始 n=9 枚举 (k=1..84)
k= 1: E= 7, T= 1
k= 2: E= 8, T= 2
k= 3: E= 9, T= 3
k= 4: E= 9, T= 4
k= 5: E=10, T= 5
k= 6: E=11, T= 7
k= 7: E=11, T= 7

```


k= 8: E=12, T=10
k= 9: E=12, T=10
k=10: E=12, T=10
k=11: E=14, T=11
k=12: E=15, T=12
k=13: E=15, T=13
k=14: E=16, T=14
k=15: E=16, T=16
k=16: E=16, T=16
k=17: E=17, T=17
k=18: E=17, T=20
k=19: E=17, T=20
k=20: E=17, T=20
k=21: E=18, T=21
k=22: E=19, T=22
k=23: E=19, T=23
k=24: E=20, T=26
k=25: E=20, T=26
k=26: E=20, T=26
k=27: E=21, T=30
k=28: E=21, T=30
k=29: E=21, T=30
k=30: E=21, T=30
k=31: E=22, T=35
k=32: E=22, T=35
k=33: E=22, T=35
k=34: E=22, T=35
k=35: E=22, T=35
k=36: E=24, T=36
k=37: E=25, T=37
k=38: E=25, T=38
k=39: E=26, T=39
k=40: E=26, T=40
k=41: E=26, T=41
k=42: E=27, T=42
k=43: E=27, T=45
k=44: E=27, T=45
k=45: E=27, T=45
k=46: E=28, T=46
k=47: E=28, T=50
k=48: E=28, T=50
k=49: E=28, T=50
k=50: E=28, T=50
k=51: E=29, T=51
k=52: E=29, T=56
k=53: E=29, T=56
k=54: E=29, T=56
k=55: E=29, T=56
k=56: E=29, T=56

k=57: E=30, T=57
 k=58: E=31, T=59
 k=59: E=31, T=59
 k=60: E=32, T=60
 k=61: E=32, T=62
 k=62: E=32, T=62
 k=63: E=33, T=65
 k=64: E=33, T=65
 k=65: E=33, T=65
 k=66: E=33, T=66
 k=67: E=34, T=71
 k=68: E=34, T=71
 k=69: E=34, T=71
 k=70: E=34, T=71
 k=71: E=34, T=71
 k=72: E=35, T=77
 k=73: E=35, T=77
 k=74: E=35, T=77
 k=75: E=35, T=77
 k=76: E=35, T=77
 k=77: E=35, T=77
 k=78: E=36, T=84
 k=79: E=36, T=84
 k=80: E=36, T=84
 k=81: E=36, T=84
 k=82: E=36, T=84
 k=83: E=36, T=84
 k=84: E=36, T=84

$E(9, k)$ 表:

k= 1: E= 7
 k= 2: E= 8
 k= 3: E= 9
 k= 4: E= 9
 k= 5: E=10
 k= 6: E=11
 k= 7: E=11
 k= 8: E=12
 k= 9: E=12
 k=10: E=12
 k=11: E=14
 k=12: E=15
 k=13: E=15
 k=14: E=16
 k=15: E=16
 k=16: E=16
 k=17: E=17
 k=18: E=17
 k=19: E=17

k=20: E=17
k=21: E=18
k=22: E=19
k=23: E=19
k=24: E=20
k=25: E=20
k=26: E=20
k=27: E=21
k=28: E=21
k=29: E=21
k=30: E=21
k=31: E=22
k=32: E=22
k=33: E=22
k=34: E=22
k=35: E=22
k=36: E=24
k=37: E=25
k=38: E=25
k=39: E=26
k=40: E=26
k=41: E=26
k=42: E=27
k=43: E=27
k=44: E=27
k=45: E=27
k=46: E=28
k=47: E=28
k=48: E=28
k=49: E=28
k=50: E=28
k=51: E=29
k=52: E=29
k=53: E=29
k=54: E=29
k=55: E=29
k=56: E=29
k=57: E=30
k=58: E=31
k=59: E=31
k=60: E=32
k=61: E=32
k=62: E=32
k=63: E=33
k=64: E=33
k=65: E=33
k=66: E=33
k=67: E=34
k=68: E=34

```

k=69: E=34
k=70: E=34
k=71: E=34
k=72: E=35
k=73: E=35
k=74: E=35
k=75: E=35
k=76: E=35
k=77: E=35
k=78: E=36
k=79: E=36
k=80: E=36
k=81: E=36
k=82: E=36
k=83: E=36
k=84: E=36

```

B.3 代码 B: $n = 10$ 低区间 SAT 验证（完整）

Listing 6: 代码 B: $n = 10$ 低区间 SAT 验证程序（PySAT）

```

1  #!/usr/bin/env python3
2  """
3  第一问 SAT 验证程序 (n=10, k 84)
4  证明: 对于所有  $1 \leq k \leq 84$ ,  $E(10, k) = E(9, k)$ 
5  使用 IDPool 统一管理变量, 彻底消除变量编号冲突。
6  """
7
8  from pysat.solvers import Solver
9  from pysat.card import CardEnc
10 from pysat.formula import IDPool
11 from itertools import combinations
12 import time, multiprocessing as mp, os, json
13 from math import comb
14
15 # 第一问穷举的  $E(9, k)$  表
16 E9 = [0] * 85
17 E9[1] = 7; E9[2] = 8; E9[3] = 9; E9[4] = 9; E9[5] = 10
18 E9[6] = 11; E9[7] = 11; E9[8] = 12; E9[9] = 12; E9[10] = 12
19 E9[11] = 14; E9[12] = 15; E9[13] = 15; E9[14] = 16; E9[15] = 16
20 E9[16] = 16; E9[17] = 17; E9[18] = 17; E9[19] = 17; E9[20] = 17
21 E9[21] = 18; E9[22] = 19; E9[23] = 19; E9[24] = 20; E9[25] = 20
22 E9[26] = 20; E9[27] = 21; E9[28] = 21; E9[29] = 21; E9[30] = 21
23 E9[31] = 22; E9[32] = 22; E9[33] = 22; E9[34] = 22; E9[35] = 22
24 E9[36] = 24; E9[37] = 25; E9[38] = 25; E9[39] = 26; E9[40] = 26
25 E9[41] = 26; E9[42] = 27; E9[43] = 27; E9[44] = 27; E9[45] = 27
26 E9[46] = 28; E9[47] = 28; E9[48] = 28; E9[49] = 28; E9[50] = 28

```

```

27 E9[51] = 29; E9[52] = 29; E9[53] = 29; E9[54] = 29; E9[55] = 29
28 E9[56] = 29; E9[57] = 30; E9[58] = 31; E9[59] = 31; E9[60] = 32
29 E9[61] = 32; E9[62] = 32; E9[63] = 33; E9[64] = 33; E9[65] = 33
30 E9[66] = 33; E9[67] = 34; E9[68] = 34; E9[69] = 34; E9[70] = 34
31 E9[71] = 34; E9[72] = 35; E9[73] = 35; E9[74] = 35; E9[75] = 35
32 E9[76] = 35; E9[77] = 35; E9[78] = 36; E9[79] = 36; E9[80] = 36
33 E9[81] = 36; E9[82] = 36; E9[83] = 36; E9[84] = 36
34
35 H0 = [(0,1),(0,2),(0,3),(0,4),(5,6),(7,8)]
36 H0_set = {tuple(sorted(e)) for e in H0}
37
38 CACHE_FILE = "sat_cache_10_v2.json"
39
40 def max_triangles(e):
41     """Kruskal-Katona 上界：给定总边数 e，无限制图的最大三角形数"""
42     if e < 6: return 0
43     a1 = 0
44     while comb(a1 + 1, 2) <= e: a1 += 1
45     rem = e - comb(a1, 2)
46     if rem == 0: return comb(a1, 3)
47     else: return comb(a1, 3) + comb(rem, 2)
48
49 def build_and_solve(k, target_edges):
50     """使用 IDPool 统一管理变量，构建 SAT 实例并求解"""
51     n = 10
52     vpool = IDPool()
53     solver = Solver()
54
55     edges = list(combinations(range(n), 2))
56
57     # 可选边变量
58     edge_var = {}
59     for e in edges:
60         if e not in H0_set:
61             edge_var[e] = vpool.id(("edge", e))
62
63     extra = target_edges - 6
64     if extra < 0 or extra > len(edge_var):
65         solver.delete()
66         return False
67
68     # 边数精确约束（使用 vpool 避免变量冲突）
69     if extra == len(edge_var):
70         for v in edge_var.values():
71             solver.add_clause([v])

```

```

72     else:
73         cnf_atmost = CardEnc.atmost(lits=list(edge_var.values()), bound=extra, vpool=
vpool)
74         cnf_atleast = CardEnc.atleast(lits=list(edge_var.values()), bound=extra, vpool=
vpool)
75         solver.append_formula(cnf_atmost.clauses)
76         solver.append_formula(cnf_atleast.clauses)
77
78     def optional_edge_var(u, v):
79         e = tuple(sorted((u, v)))
80         if e in H0_set:
81             return None
82         return edge_var[e]
83
84     tri_indicators = []
85     for x, y, z in combinations(range(n), 3):
86         opt_vars = [v for v in (
87             optional_edge_var(x, y),
88             optional_edge_var(x, z),
89             optional_edge_var(y, z)
90         ) if v is not None]
91
92         if not opt_vars:
93             continue
94
95         # 使用 vpool 创建三角形指示变量
96         act = vpool.id(("tri", x, y, z))
97         for v in opt_vars:
98             solver.add_clause([-act, v])
99         solver.add_clause([act] + [-v for v in opt_vars])
100         tri_indicators.append(act)
101
102     if k > len(tri_indicators):
103         solver.delete()
104         return False
105
106     # 三角形数至少为 k (使用 vpool 避免变量冲突)
107     if k == len(tri_indicators):
108         for act in tri_indicators:
109             solver.add_clause([act])
110     else:
111         cnf_tri = CardEnc.atleast(lits=tri_indicators, bound=k, vpool=vpool)
112         solver.append_formula(cnf_tri.clauses)
113
114     sat = solver.solve()

```

```

115     solver.delete()
116     return sat
117
118 def solve_k(k):
119     E9k = E9[k]
120     for e in range(6, E9k):
121         if max_triangles(e) < k: # 理论上界排除
122             continue
123         if build_and_solve(k, e):
124             return (k, e, True)
125     return (k, None, False)
126
127 def main():
128     tasks = list(range(1, 85))
129     cache = {}
130     if os.path.exists(CACHE_FILE):
131         with open(CACHE_FILE, 'r') as f:
132             cache = json.load(f)
133
134     new_tasks = [k for k in tasks if str(k) not in cache]
135     print(f"待验证: {len(new_tasks)} 个 k 值")
136
137     with mp.Pool(processes=8) as pool:
138         for k, e, found in pool.imap_unordered(solve_k, new_tasks):
139             if found:
140                 print(f" 发现反例: k={k}, e={e}")
141                 return
142             else:
143                 print(f" k={k:2d}: 验证通过")
144                 cache[str(k)] = "PASS"
145                 with open(CACHE_FILE, 'w') as f:
146                     json.dump(cache, f)
147
148     print("所有 k 值验证通过。结论:  $E(10, k) = E(9, k)$  对所有  $1 \leq k \leq 84$  成立。")
149
150 if __name__ == "__main__":
151     main()

```

实现注记。代码 B 中求解器初始化为 `Solver()` (PySAT 默认后端 m22)。正式运行环境 (见附录 A 计算环境) 使用的是 CaDiCaL 1.9.5 后端 (`Solver(name='cadical195')`)；实测默认 m22 后端在部分实例上超出合理时限，且与多进程组合时存在偶发崩溃，故复现时请改用 cadical195。本附录 B.4 的输出即由 CaDiCaL 1.9.5 后端产生。

B.4 代码 B 运行输出 (完整)

Listing 7: 代码 B 完整运行输出

待验证：84 个 k 值

$k=1$: 验证通过
 $k=2$: 验证通过
 $k=3$: 验证通过
 $k=4$: 验证通过
 $k=5$: 验证通过
 $k=6$: 验证通过
 $k=7$: 验证通过
 $k=8$: 验证通过
 $k=9$: 验证通过
 $k=11$: 验证通过
 $k=10$: 验证通过
 $k=12$: 验证通过
 $k=13$: 验证通过
 $k=14$: 验证通过
 $k=15$: 验证通过
 $k=16$: 验证通过
 $k=19$: 验证通过
 $k=21$: 验证通过
 $k=20$: 验证通过
 $k=22$: 验证通过
 $k=17$: 验证通过
 $k=18$: 验证通过
 $k=23$: 验证通过
 $k=24$: 验证通过
 $k=25$: 验证通过
 $k=26$: 验证通过
 $k=27$: 验证通过
 $k=28$: 验证通过
 $k=30$: 验证通过
 $k=31$: 验证通过
 $k=33$: 验证通过
 $k=32$: 验证通过
 $k=34$: 验证通过
 $k=35$: 验证通过
 $k=29$: 验证通过
 $k=36$: 验证通过
 $k=37$: 验证通过
 $k=39$: 验证通过
 $k=40$: 验证通过
 $k=41$: 验证通过
 $k=38$: 验证通过
 $k=42$: 验证通过
 $k=44$: 验证通过
 $k=45$: 验证通过
 $k=43$: 验证通过
 $k=46$: 验证通过
 $k=50$: 验证通过

k=48: 验证通过
 k=51: 验证通过
 k=57: 验证通过
 k=58: 验证通过
 k=59: 验证通过
 k=60: 验证通过
 k=61: 验证通过
 k=62: 验证通过
 k=63: 验证通过
 k=64: 验证通过
 k=65: 验证通过
 k=66: 验证通过
 k=52: 验证通过
 k=68: 验证通过
 k=69: 验证通过
 k=70: 验证通过
 k=71: 验证通过
 k=72: 验证通过
 k=67: 验证通过
 k=73: 验证通过
 k=75: 验证通过
 k=74: 验证通过
 k=76: 验证通过
 k=77: 验证通过
 k=78: 验证通过
 k=79: 验证通过
 k=80: 验证通过
 k=81: 验证通过
 k=82: 验证通过
 k=83: 验证通过
 k=84: 验证通过
 k=47: 验证通过
 k=49: 验证通过
 k=55: 验证通过
 k=54: 验证通过
 k=56: 验证通过
 k=53: 验证通过

所有 k 值验证通过。结论： $E(10, k) = E(9, k)$ 对所有 $1 \leq k \leq 84$ 成立。

B.5 代码 C: $n \geq 11$ 归约后 SAT 验证 (完整)

Listing 8: 代码 C: $n \geq 11$ 归约后 SAT 验证程序 (PySAT + CaDiCaL 1.9.5)

```

1 #!/usr/bin/env python3
2 """
3 第二问 SAT 验证程序 (最终版)
4 定理 2.1 的计算机辅助验证。
5 运行前请删除旧缓存文件 sat_cache_q2_final.json。

```

```

6  """
7
8  from pysat.solvers import Solver
9  from pysat.card import CardEnc, EncType
10 from pysat.formula import IDPool
11 from itertools import combinations
12 import time, multiprocessing as mp, os, json
13 from math import comb
14
15 # ----- 配置 -----
16 CACHE_FILE = "sat_cache_q2_final.json"
17 CACHE_VERSION = "q2-final-20250722"
18 SOLVER_NAME = "cadical195" # 正式运行使用 CaDiCaL 1.9.5
19 TRI_ENCODING = EncType.cardnetwrk # 三角形基数编码
20 EDGE_ENCODING = EncType.seqcounter # 边数基数编码
21 TRI_ENC_NAME = "cardnetwrk"
22 EDGE_ENC_NAME = "seqcounter"
23 # -----
24
25 E9 = [0]*85
26 E9[1] = 7; E9[2] = 8; E9[3] = 9; E9[4] = 9; E9[5] = 10
27 E9[6] = 11; E9[7] = 11; E9[8] = 12; E9[9] = 12; E9[10] = 12
28 E9[11] = 14; E9[12] = 15; E9[13] = 15; E9[14] = 16; E9[15] = 16
29 E9[16] = 16; E9[17] = 17; E9[18] = 17; E9[19] = 17; E9[20] = 17
30 E9[21] = 18; E9[22] = 19; E9[23] = 19; E9[24] = 20; E9[25] = 20
31 E9[26] = 20; E9[27] = 21; E9[28] = 21; E9[29] = 21; E9[30] = 21
32 E9[31] = 22; E9[32] = 22; E9[33] = 22; E9[34] = 22; E9[35] = 22
33 E9[36] = 24; E9[37] = 25; E9[38] = 25; E9[39] = 26; E9[40] = 26
34 E9[41] = 26; E9[42] = 27; E9[43] = 27; E9[44] = 27; E9[45] = 27
35 E9[46] = 28; E9[47] = 28; E9[48] = 28; E9[49] = 28; E9[50] = 28
36 E9[51] = 29; E9[52] = 29; E9[53] = 29; E9[54] = 29; E9[55] = 29
37 E9[56] = 29; E9[57] = 30; E9[58] = 31; E9[59] = 31; E9[60] = 32
38 E9[61] = 32; E9[62] = 32; E9[63] = 33; E9[64] = 33; E9[65] = 33
39 E9[66] = 33; E9[67] = 34; E9[68] = 34; E9[69] = 34; E9[70] = 34
40 E9[71] = 34; E9[72] = 35; E9[73] = 35; E9[74] = 35; E9[75] = 35
41 E9[76] = 35; E9[77] = 35; E9[78] = 36; E9[79] = 36; E9[80] = 36
42 E9[81] = 36; E9[82] = 36; E9[83] = 36; E9[84] = 36
43
44 H0 = [(0,1),(0,2),(0,3),(0,4),(5,6),(7,8)]
45 H0_set = {tuple(sorted(e)) for e in H0}
46
47 def make_solver():
48     return Solver(name=SOLVER_NAME)
49
50 def verify_witness(k, m, chosen_edges):

```

```

51     n = 9 + m
52     edge_set = set(H0_set) | chosen_edges
53     assert len(edge_set) < E9[k], f"边数 {len(edge_set)} 不小于 E9[{k}]=E9[k]"
54     tri = 0
55     tri_by_vert = [0]*n
56     for x, y, z in combinations(range(n), 3):
57         if ((x,y) in edge_set or (y,x) in edge_set) and \
58             ((x,z) in edge_set or (z,x) in edge_set) and \
59             ((y,z) in edge_set or (z,y) in edge_set):
60             tri += 1
61             tri_by_vert[x] += 1; tri_by_vert[y] += 1; tri_by_vert[z] += 1
62     assert tri >= k, f"三角形数 {tri} < k={k}"
63     for u in range(9, n):
64         assert tri_by_vert[u] >= 1, f"自由点 {u} 不在任何三角形中"
65     return tri
66
67 def build_and_solve(k, m):
68     n = 9 + m
69     vpool = IDPool()
70     solver = make_solver()
71     edges = [(i,j) for i in range(n) for j in range(i+1, n)]
72     edge_var = {}
73     for e in edges:
74         if e not in H0_set:
75             edge_var[e] = vpool.id(("edge", e))
76     all_vars = list(edge_var.values())
77     max_extra = E9[k] - 7
78     if max_extra < 0:
79         solver.delete()
80         return False, None
81     cnf_edges = CardEnc.atmost(lits=all_vars, bound=max_extra,
82                               vpool=vpool, encoding=EDGE_ENCODING)
83     solver.append_formula(cnf_edges.clauses)
84     free_triActs = {u: [] for u in range(9, n)}
85     def edge_exists(ed):
86         if ed in H0_set: return None
87         return edge_var[ed]
88     tri_inds = []
89     for x, y, z in combinations(range(n), 3):
90         e1 = edge_exists(tuple(sorted((x,y))))
91         e2 = edge_exists(tuple(sorted((x,z))))
92         e3 = edge_exists(tuple(sorted((y,z))))
93         opt_vars = [v for v in (e1,e2,e3) if v is not None]
94         if not opt_vars: continue
95         act = vpool.id(("tri", x, y, z))

```

```

96         for v in opt_vars: solver.add_clause([-act, v])
97         solver.add_clause([act] + [-v for v in opt_vars])
98         tri_inds.append(act)
99         for u in (x, y, z):
100             if u >= 9: free_tri_acts[u].append(act)
101     if k > len(tri_inds):
102         solver.delete()
103         return False, None
104     if k == len(tri_inds):
105         for act in tri_inds: solver.add_clause([act])
106     else:
107         cnf_tri = CardEnc.atleast(lits=tri_inds, bound=k,
108                                   vpool=vpool, encoding=TRI_ENCODING)
109         solver.append_formula(cnf_tri.clauses)
110     for u in range(9, n):
111         assert free_tri_acts[u], f"自由点 {u} 无候选三角形"
112         solver.add_clause(free_tri_acts[u])
113     sat = solver.solve()
114     witness = None
115     if sat:
116         model = {v for v in solver.get_model() if v > 0}
117         witness = {e for e, var in edge_var.items() if var in model}
118     solver.delete()
119     return sat, witness
120
121 def solve_task(args):
122     k, m = args
123     sat, witness = build_and_solve(k, m)
124     if sat:
125         verify_witness(k, m, witness)
126         return (k, m, True, witness)
127     return (k, m, False, None)
128
129 def main():
130     tasks = []
131     for k in range(1, 85):
132         max_m = E9[k] - 7
133         for m in range(2, max_m+1):
134             tasks.append((k, m))
135     assert len(tasks) == 1426, f"任务总数应为 1426, 实际 {len(tasks)}"
136     cache = {}
137     if os.path.exists(CACHE_FILE):
138         with open(CACHE_FILE, "r", encoding="utf-8") as f:
139             cache = json.load(f)
140             expected_meta = {

```

```

141     "version": CACHE_VERSION,
142     "solver": SOLVER_NAME,
143     "edge_encoding": EDGE_ENC_NAME,
144     "tri_encoding": TRI_ENC_NAME,
145     "expected_tasks": 1426
146 }
147 if cache.get("meta") != expected_meta:
148     print("缓存元数据不匹配, 忽略旧缓存")
149     cache = {}
150 if "results" not in cache:
151     cache["results"] = {}
152 cache["meta"] = {
153     "version": CACHE_VERSION,
154     "solver": SOLVER_NAME,
155     "edge_encoding": EDGE_ENC_NAME,
156     "tri_encoding": TRI_ENC_NAME,
157     "expected_tasks": 1426
158 }
159 new_tasks = [(k,m) for (k,m) in tasks
160              if cache["results"].get(f"{k}_{m}") != "PASS"]
161 print(f"总任务数: {len(tasks)}, 待完成: {len(new_tasks)}")
162 if not new_tasks:
163     expected_keys = {f"{k}_{m}" for k, m in tasks}
164     passed_keys = {key for key, value in cache["results"].items() if value == "PASS"}
165     assert expected_keys <= passed_keys, "缓存缺少预期 PASS 任务"
166     print("缓存中的全部 1426 个预期实例均为 PASS。")
167     return
168 PROCESSES = 1
169 with mp.Pool(processes=PROCESSES) as pool:
170     for k, m, sat, witness in pool.imap_unordered(solve_task, new_tasks):
171         if sat:
172             result = {
173                 "k": k, "m": m, "n": 9+m,
174                 "forced_edges": sorted(H0_set),
175                 "chosen_edges": sorted(witness)
176             }
177             with open("counterexample.json", "w") as f:
178                 json.dump(result, f, indent=2)
179             print(f" 发现并保存反例: k={k}, m={m}")
180             pool.terminate()
181             pool.join()
182             return
183         else:
184             print(f"  k={k:2d}, m={m:2d}: UNSAT")

```

```

185         cache["results"][f"{k}_{m}"] = "PASS"
186         temp_file = CACHE_FILE + ".tmp"
187         with open(temp_file, "w", encoding="utf-8") as f:
188             json.dump(cache, f)
189             f.flush()
190             os.fsync(f.fileno())
191             os.replace(temp_file, CACHE_FILE)
192         assert all(cache["results"].get(f"{k}_{m}") == "PASS" for k, m in tasks), \
193             "任务未完全通过"
194         print("所有 1426 个实例均为 UNSAT。命题 2.1 已通过计算机辅助验证。")
195
196 if __name__ == "__main__":
197     main()

```

B.6 代码 C 运行输出

Listing 9: 代码 C 运行输出（摘录）

```

总任务数：1426，待完成：1426
k= 3, m= 2: UNSAT
k= 4, m= 2: UNSAT
k= 5, m= 2: UNSAT
...
k=84, m=29: UNSAT
所有 1426 个实例均为 UNSAT。命题 2.1 已通过计算机辅助验证。

```

说明。代码 C 的完整运行日志共 1426 行（每行对应一个 (k, m) 实例，全部为 UNSAT），随本文附电子文件；日志首尾与上表摘录一致。全部实例总数 $\sum_{k=1}^{84} \max\{E(9, k) - 8, 0\} = 1426$ 已由程序内断言复核。

B.7 代码 D：第三问下界计算程序

Listing 10: 代码 D：第三问六项理论下界与 L_{DT} 的计算（bounds.py）

```

1 from math import comb
2
3
4 def ceil_div(a, b):
5     """Return ceil(a / b), assuming b > 0."""
6     if b <= 0:
7         raise ValueError("b must be positive")
8     return -((-a) // b)
9
10
11 def validate_parameters(n, k, d):
12     if n < 9:

```

```

13         raise ValueError("n must satisfy n >= 9")
14     if not (0 <= d <= n - 1):
15         raise ValueError("d must satisfy 0 <= d <= n - 1")
16     if not (0 <= k <= comb(n, 3)):
17         raise ValueError("k must satisfy 0 <= k <= C(n, 3)")
18
19
20 def degree_lower_bounds(n, d):
21     """
22     Return degree lower bounds imposed by  $\delta(G) \geq d$  and  $H_0$ .
23
24     Forced edges:
25         01, 02, 03, 04, 56, 78.
26
27     The bounds are sorted in descending order because compute_U
28     fills coordinates with larger initial lower bounds first.
29     """
30     if n < 9:
31         raise ValueError("n must satisfy n >= 9")
32     if not (0 <= d <= n - 1):
33         raise ValueError("d must satisfy 0 <= d <= n - 1")
34
35     ell = [max(d, 4)]
36     ell.extend([max(d, 1)] * 8)
37     ell.extend([d] * (n - 9))
38
39     return sorted(ell, reverse=True)
40
41
42 def handshake_bound(n, d):
43     return ceil_div(n * d, 2)
44
45
46 def forced_degree_gap_bound(n, d):
47     gap = (
48         max(d - 4, 0)
49         + 8 * max(d - 1, 0)
50         + (n - 9) * d
51     )
52     return 6 + ceil_div(gap, 2)
53
54
55 def triangle_edge_bound(n, k):
56     return ceil_div(3 * k, n - 2)
57

```

```

58
59 def triangle_nonforced_edge_bound(n, k):
60     return 6 + ceil_div(k, n - 2)
61
62
63 def kk_triangle_max_for_edges(m):
64     """
65     Kruskal-Katona upper bound for the number of triangles.
66
67     Write  $m = C(a, 2) + b$  with  $0 \leq b < a$ .
68     The maximum number of triangles is  $C(a, 3) + C(b, 2)$ .
69     """
70     if m < 0:
71         raise ValueError("m must be nonnegative")
72
73     a = 1
74
75     while comb(a + 1, 2) <= m:
76         a += 1
77
78     b = m - comb(a, 2)
79
80     if not (0 <= b < a):
81         raise RuntimeError("invalid Kruskal-Katona decomposition")
82
83     return comb(a, 3) + comb(b, 2)
84
85
86 def compute_L_KK(n, k):
87     """
88     Compute the KK lower bound subject to  $m \leq C(n, 2)$ .
89
90     Abstractly, the KK lower bound depends on k. The parameter n is
91     used here to enforce the edge limit of an n-vertex graph.
92     """
93     if n < 1:
94         raise ValueError("n must be positive")
95     if k < 0:
96         raise ValueError("k must be nonnegative")
97
98     for m in range(comb(n, 2) + 1):
99         if kk_triangle_max_for_edges(m) >= k:
100             return m
101
102     return float("inf")

```



```

103
104
105 def compute_U(n, d, m, return_vector=False):
106     """
107     Compute the relaxed maximum
108
109      $U(n, d, m) = \max \sum_i C(x_i, 2),$ 
110
111     subject to
112
113      $\text{ell}_i \leq x_i \leq n-1,$ 
114      $\sum_i x_i = 2m.$ 
115
116     The maximizing vector need not be graphical.
117     """
118     if n < 9:
119         raise ValueError("n must satisfy n >= 9")
120     if not (0 <= d <= n - 1):
121         raise ValueError("d must satisfy 0 <= d <= n - 1")
122     if not (0 <= m <= comb(n, 2)):
123         return None
124
125     ell = degree_lower_bounds(n, d)
126     total_degree = 2 * m
127     lower_sum = sum(ell)
128
129     if total_degree < lower_sum:
130         return None
131
132     if total_degree > n * (n - 1):
133         return None
134
135     x = ell[:]
136     remaining = total_degree - lower_sum
137
138     # ell is descending, so larger lower bounds are filled first.
139     for i in range(n):
140         capacity = n - 1 - x[i]
141         added = min(remaining, capacity)
142
143         x[i] += added
144         remaining -= added
145
146         if remaining == 0:
147             break

```

```

148
149     if remaining != 0:
150         return None
151
152     value = sum(comb(x_i, 2) for x_i in x)
153
154     if return_vector:
155         return value, x
156
157     return value
158
159
160 def compute_L_DT(n, k, d):
161     """
162     Compute
163
164     L_DT(n,k,d) = min {m : U(n,d,m) >= 3k}.
165     """
166     validate_parameters(n, k, d)
167
168     ell = degree_lower_bounds(n, d)
169
170     first_m = max(
171         6,
172         ceil_div(sum(ell), 2),
173     )
174
175     for m in range(first_m, comb(n, 2) + 1):
176         value = compute_U(n, d, m)
177
178         if value is not None and value >= 3 * k:
179             return m
180
181     return float("inf")
182
183
184 def compute_lower_bounds(n, k, d):
185     """
186     Compute the six theoretical lower bounds and their maximum.
187     """
188     validate_parameters(n, k, d)
189
190     bounds = {
191         "handshake":
192             handshake_bound(n, d),

```

```

193
194     "forced_degree_gap":
195         forced_degree_gap_bound(n, d),
196
197     "triangle_edge":
198         triangle_edge_bound(n, k),
199
200     "triangle_nonforced_edge":
201         triangle_nonforced_edge_bound(n, k),
202
203     "Kruskal_Katona":
204         compute_L_KK(n, k),
205
206     "degree_triangle":
207         compute_L_DT(n, k, d),
208 }
209
210 bounds["combined"] = max(bounds.values())
211
212 return bounds
213
214
215 def print_bounds(n, k, d):
216     print("=" * 64)
217     print(f"n={n}, k={k}, d={d}")
218
219     bounds = compute_lower_bounds(n, k, d)
220
221     for name, value in bounds.items():
222         print(f"{name:28s}: {value}")
223
224
225 def demo():
226     print_bounds(9, 11, 3)
227
228     print()
229
230     for m in (14, 15):
231         result = compute_U(
232             9,
233             3,
234             m,
235             return_vector=True,
236         )
237

```

```

238         value, vector = result
239
240         print(f"U(9,3,{m}) = {value}")
241         print(f"relaxed vector = {vector}")
242
243
244 if __name__ == "__main__":
245     demo()

```

B.8 代码 E：第三问逐层搜索程序

Listing 11: 代码 E：第三问按累计边数上限的见证图搜索（exact_search.py）

```

1  import argparse
2  from itertools import combinations
3  from math import comb
4
5  from ortools.sat.python import cp_model
6
7  from bounds import compute_lower_bounds
8
9
10 FORCED_EDGES = {
11     (0, 1),
12     (0, 2),
13     (0, 3),
14     (0, 4),
15     (5, 6),
16     (7, 8),
17 }
18
19
20 def normalize_edge(u, v):
21     if u == v:
22         raise ValueError("loops are not allowed")
23
24     return (u, v) if u < v else (v, u)
25
26
27 def check_graph(n, edges, k, d, verbose=True):
28     """
29     Independently verify a candidate graph.
30
31     Checks endpoints, loops, duplicate edges, forced edges,
32     minimum degree, and the number of triangles.

```

```

33     """
34     edge_set = set()
35
36     for u, v in edges:
37         if not (0 <= u < n and 0 <= v < n):
38             if verbose:
39                 print("Invalid endpoint:", (u, v))
40             return False
41
42         if u == v:
43             if verbose:
44                 print("Loop found:", (u, v))
45             return False
46
47         normalized = normalize_edge(u, v)
48
49         if normalized in edge_set:
50             if verbose:
51                 print("Duplicate edge:", (u, v))
52             return False
53
54         edge_set.add(normalized)
55
56     missing_forced = FORCED_EDGES - edge_set
57
58     degrees = [0] * n
59
60     for u, v in edge_set:
61         degrees[u] += 1
62         degrees[v] += 1
63
64     minimum_degree = min(degrees)
65
66     triangle_count = 0
67
68     for a, b, c in combinations(range(n), 3):
69         if (
70             (a, b) in edge_set
71             and (a, c) in edge_set
72             and (b, c) in edge_set
73         ):
74             triangle_count += 1
75
76     valid = (
77         not missing_forced

```

```

78         and minimum_degree >= d
79         and triangle_count >= k
80     )
81
82     if verbose:
83         print("Actual edge count:", len(edge_set))
84         print("Degrees:", degrees)
85         print("Minimum degree:", minimum_degree)
86         print("Triangle count:", triangle_count)
87         print("Missing forced edges:", sorted(missing_forced))
88         print("Minimum-degree condition:", minimum_degree >= d)
89         print("Triangle condition:", triangle_count >= k)
90         print("Verified:", valid)
91
92     return valid
93
94
95 def search_at_most(n, k, d, max_edges, time_limit):
96     """
97     Search for a graph satisfying
98
99     H0 subset of G,
100     delta(G) >= d,
101     t(G) >= k,
102     e(G) <= max_edges.
103     """
104     model = cp_model.CpModel()
105     all_edges = list(combinations(range(n), 2))
106
107     x = {
108         edge: model.NewBoolVar(f"x_{edge[0]}_{edge[1]}")
109         for edge in all_edges
110     }
111
112     for edge in FORCED_EDGES:
113         normalized = normalize_edge(*edge)
114
115         if normalized not in x:
116             raise ValueError(
117                 f"forced edge {normalized} is outside the vertex set"
118             )
119
120         model.Add(x[normalized] == 1)
121
122     model.Add(sum(x.values()) <= max_edges)

```

```

123
124     for v in range(n):
125         incident_edges = [
126             x[normalize_edge(u, v)]
127             for u in range(n)
128             if u != v
129         ]
130
131         model.Add(sum(incident_edges) >= d)
132
133     triangle_variables = []
134
135     for a, b, c in combinations(range(n), 3):
136         ab = x[(a, b)]
137         ac = x[(a, c)]
138         bc = x[(b, c)]
139
140         triangle = model.NewBoolVar(
141             f"triangle_{a}_{b}_{c}"
142         )
143
144         triangle_variables.append(triangle)
145
146         model.Add(triangle <= ab)
147         model.Add(triangle <= ac)
148         model.Add(triangle <= bc)
149         model.Add(triangle >= ab + ac + bc - 2)
150
151     model.Add(sum(triangle_variables) >= k)
152
153     solver = cp_model.CpSolver()
154     solver.parameters.max_time_in_seconds = time_limit
155     solver.parameters.num_search_workers = 8
156
157     status = solver.Solve(model)
158     status_name = solver.StatusName(status)
159
160     if status not in (
161         cp_model.OPTIMAL,
162         cp_model.FEASIBLE,
163     ):
164         return status_name, None
165
166     edge_list = [
167         edge

```

```

168         for edge in all_edges
169             if solver.Value(x[edge]) == 1
170         ]
171
172     return status_name, edge_list
173
174
175 def search_from_bound(n, k, d, start_bound, time_limit):
176     """
177     Search cumulative edge bounds beginning at start_bound.
178
179     INFEASIBLE at m excludes all legal graphs with e(G) <= m.
180     UNKNOWN or any unhandled status stops the search.
181     """
182     for m in range(start_bound, comb(n, 2) + 1):
183         print()
184         print("-" * 64)
185         print(f"Searching for a graph with e(G) <= {m}")
186
187         status_name, edge_list = search_at_most(
188             n=n,
189             k=k,
190             d=d,
191             max_edges=m,
192             time_limit=time_limit,
193         )
194
195         print("Solver status:", status_name)
196
197         if status_name in ("OPTIMAL", "FEASIBLE"):
198             print("A candidate witness was found.")
199             return status_name, m, edge_list
200
201         if status_name == "INFEASIBLE":
202             print(
203                 f"No graph with e(G) <= {m} exists in this model."
204             )
205             continue
206
207         if status_name == "UNKNOWN":
208             print(
209                 "The search did not finish. No feasibility or "
210                 "infeasibility conclusion can be made."
211             )
212     else:

```



```

213         print(f"Unhandled solver status: {status_name}")
214
215     return status_name, m, None
216
217     return "NOT_FOUND", None, None
218
219
220 def parse_arguments():
221     parser = argparse.ArgumentParser(
222         description=(
223             "Search for a legal graph under cumulative edge bounds."
224         )
225     )
226
227     parser.add_argument("--n", type=int, default=9)
228     parser.add_argument("--k", type=int, default=11)
229     parser.add_argument("--d", type=int, default=3)
230
231     parser.add_argument(
232         "--lower-bound",
233         type=int,
234         default=None,
235         help=(
236             "Initial cumulative edge bound. If omitted, the combined "
237             "theoretical lower bound is used."
238         ),
239     )
240
241     parser.add_argument(
242         "--time-limit",
243         type=float,
244         default=60.0,
245         help="Time limit in seconds for each cumulative bound.",
246     )
247
248     return parser.parse_args()
249
250
251 def validate_arguments(args):
252     if args.n < 9:
253         raise ValueError("n must satisfy n >= 9")
254
255     if not (0 <= args.d <= args.n - 1):
256         raise ValueError("d must satisfy 0 <= d <= n - 1")
257

```

```

258     if not (0 <= args.k <= comb(args.n, 3)):
259         raise ValueError("k must satisfy 0 <= k <= C(n,3)")
260
261     if args.time_limit <= 0:
262         raise ValueError("time-limit must be positive")
263
264     if args.lower_bound is not None:
265         if not (0 <= args.lower_bound <= comb(args.n, 2)):
266             raise ValueError(
267                 "lower-bound must satisfy "
268                 "0 <= lower-bound <= C(n,2)"
269             )
270
271
272 def main():
273     args = parse_arguments()
274     validate_arguments(args)
275
276     bounds = compute_lower_bounds(
277         args.n,
278         args.k,
279         args.d,
280     )
281
282     theoretical_lower_bound = bounds["combined"]
283
284     if args.lower_bound is None:
285         search_start = theoretical_lower_bound
286     else:
287         search_start = args.lower_bound
288
289     print("Parameters")
290     print(f"n = {args.n}")
291     print(f"k = {args.k}")
292     print(f"d = {args.d}")
293     print(
294         "Combined theoretical lower bound = "
295         f"{theoretical_lower_bound}"
296     )
297     print(f"Search starts at = {search_start}")
298     print(
299         "Time limit per cumulative bound = "
300         f"{args.time_limit} seconds"
301     )
302

```

```

303     if search_start < theoretical_lower_bound:
304         print()
305         print(
306             "Warning: the selected start is below the known "
307             "theoretical lower bound."
308         )
309
310     if search_start > theoretical_lower_bound:
311         print()
312         print(
313             "Warning: cumulative bounds between the theoretical "
314             "lower bound and the selected start will be skipped."
315         )
316
317     status_name, tested_bound, edge_list = search_from_bound(
318         n=args.n,
319         k=args.k,
320         d=args.d,
321         start_bound=search_start,
322         time_limit=args.time_limit,
323     )
324
325     print()
326     print("=" * 64)
327     print("Final solver status:", status_name)
328
329     if edge_list is None:
330         print("No independently verifiable witness was produced.")
331         return
332
333     actual_edge_count = len(edge_list)
334
335     print("Tested cumulative bound:", tested_bound)
336     print("Actual edge count:", actual_edge_count)
337     print("Edges:")
338     print(sorted(edge_list))
339
340     print()
341     print("Independent verification")
342
343     verified = check_graph(
344         n=args.n,
345         edges=edge_list,
346         k=args.k,
347         d=args.d,

```

```

348         verbose=True,
349     )
350
351     if not verified:
352         print(
353             "The candidate failed independent verification. "
354             "Do not use it as an upper bound."
355         )
356         return
357
358     print()
359     print(
360         f"Verified upper bound: "
361         f"E({args.n},{args.k},{args.d}) <= "
362         f"{actual_edge_count}"
363     )
364
365     if actual_edge_count == theoretical_lower_bound:
366         print(
367             f"E({args.n},{args.k},{args.d}) "
368             f"= {actual_edge_count}"
369         )
370     else:
371         print(
372             f"{theoretical_lower_bound} <= "
373             f"E({args.n},{args.k},{args.d}) <= "
374             f"{actual_edge_count}"
375         )
376
377
378 if __name__ == "__main__":
379     main()

```

B.9 代码 F：第三问直接优化程序

Listing 12: 代码 F：第三问直接最小化边数的 CP-SAT 求解 (exact_optimize.py)

```

1 import argparse
2 import math
3 from itertools import combinations
4 from math import comb
5
6 from ortools.sat.python import cp_model
7
8 from bounds import compute_lower_bounds

```

```

9  from exact_search import (
10     FORCED_EDGES,
11     check_graph,
12     normalize_edge,
13 )
14
15
16 def optimize_graph(n, k, d, time_limit, workers):
17     """
18     Minimize e(G) subject to
19
20         H0 subset of G,
21         delta(G) >= d,
22         t(G) >= k.
23     """
24     model = cp_model.CpModel()
25     all_edges = list(combinations(range(n), 2))
26
27     x = {
28         edge: model.NewBoolVar(f"x_{edge[0]}_{edge[1]}")
29         for edge in all_edges
30     }
31
32     for edge in FORCED_EDGES:
33         normalized = normalize_edge(*edge)
34
35         if normalized not in x:
36             raise ValueError(
37                 f"forced edge {normalized} is outside the vertex set"
38             )
39
40         model.Add(x[normalized] == 1)
41
42     for v in range(n):
43         incident_edges = [
44             x[normalize_edge(u, v)]
45             for u in range(n)
46             if u != v
47         ]
48
49         model.Add(sum(incident_edges) >= d)
50
51     triangle_variables = []
52
53     for a, b, c in combinations(range(n), 3):

```

```

54     ab = x[(a, b)]
55     ac = x[(a, c)]
56     bc = x[(b, c)]
57
58     triangle = model.NewBoolVar(
59         f"triangle_{a}_{b}_{c}"
60     )
61
62     triangle_variables.append(triangle)
63
64     model.Add(triangle <= ab)
65     model.Add(triangle <= ac)
66     model.Add(triangle <= bc)
67     model.Add(triangle >= ab + ac + bc - 2)
68
69     model.Add(sum(triangle_variables) >= k)
70
71     edge_count = sum(x.values())
72     model.Minimize(edge_count)
73
74     solver = cp_model.CpSolver()
75     solver.parameters.max_time_in_seconds = time_limit
76     solver.parameters.num_search_workers = workers
77
78     status = solver.Solve(model)
79     status_name = solver.StatusName(status)
80
81     result = {
82         "status": status_name,
83         "edges": None,
84         "objective": None,
85         "solver_lower_bound": None,
86     }
87
88     if status in (
89         cp_model.OPTIMAL,
90         cp_model.FEASIBLE,
91     ):
92         edges = [
93             edge
94             for edge in all_edges
95             if solver.Value(x[edge]) == 1
96         ]
97
98         result["edges"] = edges

```

```

99         result["objective"] = int(
100             round(solver.ObjectiveValue())
101         )
102
103         result["solver_lower_bound"] = math.ceil(
104             solver.BestObjectiveBound() - 1e-9
105         )
106
107     return result
108
109
110 def parse_arguments():
111     parser = argparse.ArgumentParser(
112         description=(
113             "Directly minimize the number of edges in a legal graph."
114         )
115     )
116
117     parser.add_argument("--n", type=int, default=9)
118     parser.add_argument("--k", type=int, default=11)
119     parser.add_argument("--d", type=int, default=3)
120
121     parser.add_argument(
122         "--time-limit",
123         type=float,
124         default=300.0,
125         help="Total solver time limit in seconds.",
126     )
127
128     parser.add_argument(
129         "--workers",
130         type=int,
131         default=8,
132         help="Number of CP-SAT search workers.",
133     )
134
135     return parser.parse_args()
136
137
138 def validate_arguments(args):
139     if args.n < 9:
140         raise ValueError("n must satisfy n >= 9")
141
142     if not (0 <= args.d <= args.n - 1):
143         raise ValueError("d must satisfy 0 <= d <= n - 1")

```

```
144
145     if not (0 <= args.k <= comb(args.n, 3)):
146         raise ValueError("k must satisfy 0 <= k <= C(n,3)")
147
148     if args.time_limit <= 0:
149         raise ValueError("time-limit must be positive")
150
151     if args.workers <= 0:
152         raise ValueError("workers must be positive")
153
154
155 def main():
156     args = parse_arguments()
157     validate_arguments(args)
158
159     theoretical_bounds = compute_lower_bounds(
160         args.n,
161         args.k,
162         args.d,
163     )
164
165     theoretical_lower_bound = theoretical_bounds["combined"]
166
167     print("Parameters")
168     print(f"n = {args.n}")
169     print(f"k = {args.k}")
170     print(f"d = {args.d}")
171     print(
172         "Theoretical lower bound = "
173         f"{theoretical_lower_bound}"
174     )
175     print(f"Time limit = {args.time_limit} seconds")
176     print(f"Workers = {args.workers}")
177     print()
178
179     result = optimize_graph(
180         n=args.n,
181         k=args.k,
182         d=args.d,
183         time_limit=args.time_limit,
184         workers=args.workers,
185     )
186
187     status = result["status"]
188
```



```

189     print("Solver status:", status)
190
191     if status == "INFEASIBLE":
192         print(
193             "The model was declared infeasible. In the stated "
194             "parameter range, K_n should be feasible, so inputs "
195             "and the model should be checked."
196         )
197         return
198
199     if status not in ("OPTIMAL", "FEASIBLE"):
200         print(
201             "The solver produced no witness. "
202             "No new upper bound is available."
203         )
204         return
205
206     edges = result["edges"]
207     objective = result["objective"]
208     solver_lower_bound = result["solver_lower_bound"]
209
210     print("Solver objective:", objective)
211     print("Solver lower bound:", solver_lower_bound)
212     print("Edges:")
213     print(sorted(edges))
214     print()
215
216     verified = check_graph(
217         n=args.n,
218         edges=edges,
219         k=args.k,
220         d=args.d,
221         verbose=True,
222     )
223
224     if not verified:
225         print(
226             "The candidate failed independent verification. "
227             "Do not use it as an upper bound."
228         )
229         return
230
231     actual_edge_count = len(edges)
232
233     combined_lower_bound = max(

```

```

234         theoretical_lower_bound,
235         solver_lower_bound,
236     )
237
238     print()
239     print(
240         f"Verified upper bound: "
241         f"E({args.n},{args.k},{args.d}) <= "
242         f"{actual_edge_count}"
243     )
244
245     print(
246         f"Current lower bound: "
247         f"E({args.n},{args.k},{args.d}) >= "
248         f"{combined_lower_bound}"
249     )
250
251     if status == "OPTIMAL":
252         print()
253         print(
254             "CP-SAT completed the optimization and returned OPTIMAL."
255         )
256         print(
257             f"Computational result: "
258             f"E({args.n},{args.k},{args.d}) "
259             f"= {actual_edge_count}"
260         )
261
262         if actual_edge_count == theoretical_lower_bound:
263             print(
264                 "The witness attains the pure theoretical lower "
265                 "bound. Hence the equality follows from the "
266                 "theoretical bound and the verified witness."
267             )
268         else:
269             print(
270                 "The lower-bound step beyond the theoretical bound "
271                 "depends on CP-SAT's completed optimality proof."
272             )
273
274     elif combined_lower_bound == actual_edge_count:
275         print()
276         print(
277             "The solver lower bound and verified upper bound coincide."
278         )

```

```

279     print(
280         f"Computationally, "
281         f"E({args.n},{args.k},{args.d}) "
282         f"= {actual_edge_count}"
283     )
284
285     else:
286         print()
287         print("Current interval:")
288         print(
289             f"{combined_lower_bound} <= "
290             f"E({args.n},{args.k},{args.d}) <= "
291             f"{actual_edge_count}"
292         )
293         print(
294             "The optimization stopped before proving optimality. "
295             "Increasing the time limit may tighten the interval."
296         )
297
298
299 if __name__ == "__main__":
300     main()

```

B.10 代码 G：第四问 $F(n, k)$ 精确求解器

Listing 13: 代码 G：第四问连通且三角形边不交的 $F(n, k)$ CP-SAT 精确求解 (q4_f_solver.py)

```

1  #!/usr/bin/env python3
2  """
3  第四问：连通且三角形边不交的最少边数 F(n,k) 精确求解器
4  用法：直接运行，按提示输入 n,k（逗号分隔），例如 9,5 或 9,5,600
5  依赖：ortools>=9.0
6  """
7
8  import sys
9  from collections import deque
10 from itertools import combinations
11 from ortools.sat.python import cp_model
12
13 # 强制子图 H0（全部升序排列）
14 FORCED_EDGES = [(0, 1), (0, 2), (0, 3), (0, 4), (5, 6), (7, 8)]
15
16 def verify_solution(n, k, edges):
17     """独立验证边集是否满足全部条件"""
18     # 统一边为升序

```

```

19 def norm(e):
20     return (e[0], e[1]) if e[0] < e[1] else (e[1], e[0])
21 edges = [norm(e) for e in edges]
22 edge_set = set(edges)
23
24 # 1. 强制边检查
25 for u, v in FORCED_EDGES:
26     if (u, v) not in edge_set:
27         return False, f"强制边 ({u},{v}) 缺失"
28
29 # 2. 连通性检查 (BFS)
30 adj = [[] for _ in range(n)]
31 for u, v in edge_set:
32     adj[u].append(v)
33     adj[v].append(u)
34 visited = [False] * n
35 queue = deque([0])
36 visited[0] = True
37 while queue:
38     u = queue.popleft()
39     for v in adj[u]:
40         if not visited[v]:
41             visited[v] = True
42             queue.append(v)
43 if not all(visited):
44     unvisited = [i for i, vis in enumerate(visited) if not vis]
45     return False, f"图不连通, 未访问顶点: {unvisited}"
46
47 # 3. 三角形边不交检查 & 统计三角形数
48 edge_triangle_count = {e: 0 for e in edge_set}
49 triangle_count = 0
50 for a, b, c in combinations(range(n), 3):
51     if (a, b) in edge_set and (a, c) in edge_set and (b, c) in edge_set:
52         triangle_count += 1
53         edge_triangle_count[(a, b)] += 1
54         edge_triangle_count[(a, c)] += 1
55         edge_triangle_count[(b, c)] += 1
56 for e, cnt in edge_triangle_count.items():
57     if cnt > 1:
58         return False, f"边 {e} 被 {cnt} 个三角形共享, 违反边不交"
59
60 # 4. 三角形数检查
61 if triangle_count < k:
62     return False, f"三角形数 {triangle_count} < k={k}"
63

```

```

64     return True, f"验证通过：边数={len(edge_set)}, 三角形数={triangle_count}, 连通, 强制
        边完整, 边不交"
65
66 def solve_F(n, k, time_limit=300):
67     """
68     求解 F(n,k)。返回 (status, edge_count, edges)
69     status: 'OPTIMAL', 'FEASIBLE', 'INFEASIBLE', 'UNKNOWN'
70     """
71     if n < 9:
72         raise ValueError("n 必须 >= 9")
73     if k <= 0:
74         raise ValueError("k 必须 >= 1")
75
76     # 最大边不交三角形数上界（考虑  $n \bmod 6$  的修正）
77     # 当  $n \equiv 1, 3 \pmod{6}$  时上界为  $\lfloor n(n-1)/6 \rfloor$  且可达（Steiner 三元系）；
78     # 当  $n \equiv 5 \pmod{6}$  时实际最大值为上界减 1；
79     # 其余同余类上界仍为  $\lfloor n(n-1)/6 \rfloor$ ，但并非全部可达。
80     # 此处使用保守上界仅用于快速剪枝，不影响正确性（过高估计时求解器会完整判定）。
81     def max_tri(n):
82         base = (n * (n - 1)) // 6
83         return base - 1 if n % 6 == 5 else base
84     if k > max_tri(n):
85         return "INFEASIBLE", None, None
86
87     model = cp_model.CpModel()
88     all_edges = list(combinations(range(n), 2))
89     edge_vars = {e: model.NewBoolVar(f'e_{e[0]}_{e[1]}') for e in all_edges}
90
91     # 强制边
92     for u, v in FORCED_EDGES:
93         model.Add(edge_vars[(u, v)] == 1)
94
95     # 连通性：单商品流，容量  $n-1$ 
96     flow_vars = {}
97     for u, v in all_edges:
98         flow_vars[(u, v)] = model.NewIntVar(0, n-1, f'f_{u}_{v}')
99         flow_vars[(v, u)] = model.NewIntVar(0, n-1, f'f_{v}_{u}')
100         model.Add(flow_vars[(u, v)] + flow_vars[(v, u)] <= (n-1) * edge_vars[(u, v)])
101
102     for node in range(1, n):
103         in_flows = []
104         out_flows = []
105         for neighbor in range(n):
106             if neighbor == node:
107                 continue

```

```

108         # 由于 flow_vars 包含所有方向，直接使用即可
109         in_flows.append(flow_vars[(neighbor, node)]) # neighbor -> node
110         out_flows.append(flow_vars[(node, neighbor)]) # node -> neighbor
111         model.Add(sum(in_flows) - sum(out_flows) == 1)
112
113     # 三角形变量
114     triangle_vars = {}
115     for a, b, c in combinations(range(n), 3):
116         tri = model.NewBoolVar(f'tri_{a}_{b}_{c}')
117         e_ab = edge_vars[(a, b)]
118         e_ac = edge_vars[(a, c)]
119         e_bc = edge_vars[(b, c)]
120         model.Add(tri <= e_ab)
121         model.Add(tri <= e_ac)
122         model.Add(tri <= e_bc)
123         model.Add(tri >= e_ab + e_ac + e_bc - 2)
124         triangle_vars[(a, b, c)] = tri
125
126     # 边不交：每条边至多属于一个三角形
127     edge_to_tris = {e: [] for e in all_edges}
128     for (a, b, c), tri in triangle_vars.items():
129         edge_to_tris[(a, b)].append(tri)
130         edge_to_tris[(a, c)].append(tri)
131         edge_to_tris[(b, c)].append(tri)
132     for e, tris in edge_to_tris.items():
133         if tris:
134             model.Add(sum(tris) <= 1)
135
136     model.Add(sum(triangle_vars.values()) >= k)
137     model.Minimize(sum(edge_vars.values()))
138
139     solver = cp_model.CpSolver()
140     solver.parameters.max_time_in_seconds = time_limit
141     solver.parameters.num_search_workers = 8
142     solver.parameters.log_search_progress = False
143
144     status = solver.Solve(model)
145     status_name = solver.StatusName(status)
146     if status in (cp_model.OPTIMAL, cp_model.FEASIBLE):
147         edges = [(u, v) for (u, v), var in edge_vars.items() if solver.Value(var) == 1]
148         return status_name, int(solver.ObjectiveValue()), edges
149     else:
150         return status_name, None, None
151
152 def main():

```

```

153 print("=== 第四问: F(n,k) 精确求解器 ===")
154 print("请输入 n,k (用逗号分隔, 可附加时间限制秒数, 例如 9,5 或 9,5,600): ")
155 try:
156     parts = input().strip().split(',')
157     n = int(parts[0])
158     k = int(parts[1])
159     time_limit = float(parts[2]) if len(parts) >= 3 else 300.0
160 except:
161     print("输入格式错误")
162     return
163
164 if n < 9 or k <= 0:
165     print("n 必须 >=9, k >=1")
166     return
167
168 print(f"求解 F({n},{k}), 时间限制 {time_limit} 秒...")
169 print(f"完全图边数: {n*(n-1)//2}, 总三角形数: {n*(n-1)*(n-2)//6}")
170 print(f"边不交三角形松上界: {n*(n-1)//6}")
171 status, val, edges = solve_F(n, k, time_limit)
172 print(f"状态: {status}")
173 if status == "OPTIMAL":
174     print(f" F({n},{k}) = {val}")
175 elif status == "FEASIBLE":
176     print(f" F({n},{k}) {val}")
177 elif status == "INFEASIBLE":
178     print(f"F({n},{k}) = ∞")
179 else:
180     print("未完成")
181
182 if edges is not None:
183     print(f"边集: {sorted(edges)}")
184     ok, msg = verify_solution(n, k, edges)
185     print(f"验证: {' ' if ok else ' '} {msg}")
186
187 if __name__ == "__main__":
188     main()

```

代码 G 运行示例 ($n = 9, k = 6$, 输入 9,6,1200):

```

=== 第四问: F(n,k) 精确求解器 ===
请输入 n,k (用逗号分隔, 可附加时间限制秒数, 例如 9,5 或 9,5,600):
9,6,1200
求解 F(9,6), 时间限制 1200.0 秒...
完全图边数: 36, 总三角形数: 84
边不交三角形松上界: 12
状态: OPTIMAL
F(9,6) = 18

```

边集: [(0, 1), (0, 2), (0, 3), (0, 4), (1, 2), (1, 7), (1, 8), (2, 5), (2, 6), (3, 4), (3, 5),
(3, 8), (4, 6), (4, 7), (5, 6), (5, 8), (6, 7), (7, 8)]

验证: 验证通过: 边数=18, 三角形数=6, 连通, 强制边完整, 边不交